United States Patent 12413746
Kind Code B2
Date of Patent September 09, 2025
Inventor(s) Li; Jing Ya et al.

Image encoder, image decoder, image encoding method, and image decoding method

Abstract

An image encoder is provided including circuitry and a memory coupled to the circuitry. The circuitry, in operation, responds to a size of a block satisfying a size condition by generating a prediction image using a prediction mode selected from a plurality of prediction modes. The plurality of prediction modes include a first prediction mode in which a prediction process uses a motion vector and a reference block in a same picture as the block. The circuitry encodes the block using the prediction image.

Inventors: **Li; Jing Ya (Singapore, SG), Lim; Chong Soon (Singapore, SG), Teo; Han Boon (Singapore, SG), Sun; Hai Wei (Singapore, SG), Kuo; Che Wei (Singapore, SG), Abe; Kiyofumi (Osaka, JP), Nishi; Takahiro (Nara, JP), Toma; Tadamasa (Osaka, JP), Kato; Yusuke (Osaka, JP)**

Applicant: **Panasonic Intellectual Property Corporation of America** (Torrance, CA)

Family ID: **69770990**

Assignee: **Panasonic Intellectual Property Corporation of America (Torrance, CA)**

Appl. No.: **18/679122**

Filed: **May 30, 2024**

Prior Publication Data

Document Identifier	Publication Date
US 20240323400 A1	Sep. 26, 2024

Related U.S. Application Data

continuation parent-doc US 18171242 20230217 US 12034938 child-doc US 18679122
continuation parent-doc US 16795408 20200219 US 11611758 20230321 child-doc US 18171242
us-provisional-application US 62807923 20190220

Publication Classification

Int. Cl.: **G06V10/00** (20220101); **H04N19/126** (20140101); **H04N19/156** (20140101); **H04N19/159** (20140101); **H04N19/18** (20140101); **H04N19/52** (20140101); **H04N19/61** (20140101)

U.S. Cl.:

CPC **H04N19/159** (20141101); **H04N19/126** (20141101); **H04N19/156** (20141101); **H04N19/18** (20141101); **H04N19/52** (20141101); **H04N19/61** (20141101);

Field of Classification Search

CPC: H04N (19/107); H04N (19/159); H04N (19/11); H04N (19/126); H04N (19/134); H04N (19/156); H04N (19/174); H04N (19/18); H04N (19/503); H04N (19/52); H04N (19/593); H04N (19/61); H04N (19/70)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
6097842	12/1999	Suzuki et al.	N/A	N/A
6959041	12/2004	Ishiyama	N/A	N/A
7079579	12/2005	Han et al.	N/A	N/A
7333660	12/2007	Koto	375/E7.146	H04N 19/61
7474699	12/2008	Kobayashi et al.	N/A	N/A
7792193	12/2009	Tanizawa	375/240.18	H04N 19/126
8149910	12/2011	Tanizawa	375/240.27	H04N 19/117
8594178	12/2012	Li et al.	N/A	N/A
8913662	12/2013	Karczewicz	375/240.13	H04N 19/196
8958476	12/2014	Kim et al.	N/A	N/A
9008173	12/2014	Matsuo	375/240.12	H04N 19/46
9106917	12/2014	Shimada	N/A	H04N 19/176
9277220	12/2015	Nakamura	N/A	N/A
9432668	12/2015	Bossen et al.	N/A	N/A
9549187	12/2016	Kimura	N/A	H04N 19/176
10356417	12/2018	Lawrence	N/A	N/A
10873759	12/2019	Sharman et al.	N/A	N/A
11051009	12/2020	Hsiang	N/A	N/A
11190744	12/2020	Fuchie et al.	N/A	N/A
11284105	12/2021	Sharman et al.	N/A	N/A
11470347	12/2021	Jeong	N/A	H04N 19/119
11611758	12/2022	Li	N/A	H04N 19/52
12034938	12/2023	Li	N/A	H04N 19/61
2004/0008774	12/2003	Takahashi et al.	N/A	N/A
2010/0027655	12/2009	Matsuo	375/E7.243	H04N 19/176
2010/0158111	12/2009	Bordes et al.	N/A	N/A
2014/0010446	12/2013	Chujoh et al.	N/A	N/A
2015/0085937	12/2014	Sekiguchi et al.	N/A	N/A
2018/0359469	12/2017	Won et al.	N/A	N/A
2019/0281267	12/2018	Fuchie et al.	N/A	N/A
2020/0084470	12/2019	Abe et al.	N/A	N/A
2020/0267393	12/2019	Li et al.	N/A	N/A
2021/0385463	12/2020	Guruva Reddiar	N/A	H04N 19/179

OTHER PUBLICATIONS

Andersson et al., “On indication of IBC,” JCTVC-S0129, Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Meeting #19, Strasbourg, France, Oct. 17-24, 2014, 3 pages. cited by applicant

Flynn et al., “High Efficiency Video Coding (HEVC) Range Extensions text specification: Draft 5,” JCTVC-O1005_v1, Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Meeting #13, Incheon, Korea, Apr. 18-26, 2013, 341 pages. cited by applicant

International Search Report and Written Opinion, dated Apr. 15, 2020, for International Application No. PCT/JP2020/006199, 16 pages. cited by applicant

Lee et al., “AHG5: Block size restriction of intra block copy,” JCTVC-00102, Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Meeting #15, Geneva, Switzerland, Oct. 23-Nov. 1, 2013, 9 pages. cited by applicant

Rosewarne et al., “Non-RCE3: On Intra block copy,” JCTVC-P0202, Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Meeting #16, San José, United States, Jan. 9-17, 2014, 3 pages. cited by applicant

Primary Examiner: Alavi; Amir

Attorney, Agent or Firm: Seed Intellectual Property Law Group LLP

Background/Summary

BACKGROUND

Technical Field

(1) This disclosure relates to video coding, and particularly to video encoding and decoding systems, components, and methods for performing a prediction function to build a prediction of a current block.

Description of the Related Art

(2) With advancements in video coding technology, from H.261 and MPEG-1 to H.264/AVC (Advanced Video Coding), MPEG-LA, H.265/HEVC (High Efficiency Video Coding) and H.266/VVC (Versatile Video Codec), there remains a constant need to provide improvements and optimizations to the video coding technology to process an ever-increasing amount of digital video data in various applications. This disclosure relates to further advancements, improvements and optimizations in video coding, particularly in a prediction function to build a prediction of a current block.

BRIEF SUMMARY

(3) According to one aspect, an image encoder is provided including circuitry and a memory coupled to the circuitry. The circuitry, in operation, responds to a size of a block satisfying a size condition by generating a prediction image using a prediction mode selected from a plurality of prediction modes. The plurality of prediction modes include a first prediction mode in which a prediction process uses a motion vector and a reference block in a same picture as the block. The circuitry encodes the block using the prediction image.

(4) According to one aspect, an image decoder includes circuitry and a memory coupled to the circuitry. The circuitry, in operation, responds to a size of a block satisfying a size condition by generating a prediction image using a prediction mode selected from a plurality of prediction modes. The plurality of prediction modes include a first prediction mode in which a prediction process uses a motion vector and a reference block in a same picture as the block. The circuitry decodes the block using the prediction image.

(5) Some implementations of embodiments of the present disclosure may improve an encoding efficiency, may simply be an encoding/decoding process, may accelerate an encoding/decoding process speed, may efficiently select appropriate components/operations used in encoding and decoding such as appropriate filter, block size, motion vector, reference picture, reference block, etc.

(6) Additional benefits and advantages of the disclosed embodiments will become apparent from the specification and drawings. The benefits and/or advantages may be individually obtained by the various embodiments and features of the specification and drawings, not all of which need to be provided in order to obtain one or more of such benefits and/or advantages.

(7) It should be noted that general or specific embodiments may be implemented as a system, a method, an integrated circuit, a computer program, a storage medium, or any selective combination thereof.

Description

BRIEF DESCRIPTION OF THE SEVERAL VIEWS OF THE DRAWINGS

(1) FIG. **1** is a block diagram illustrating a functional configuration of an encoder according to an embodiment.

(2) FIG. **2** is a flow chart indicating one example of an overall encoding process performed by the encoder.

(3) FIG. **3** is a conceptual diagram illustrating one example of block splitting.

(4) FIG. **4**A is a conceptual diagram illustrating one example of a slice configuration.

(5) FIG. **4**B is a conceptual diagram illustrating one example of a tile configuration.

(6) FIG. **5**A is a chart indicating transform basis functions for various transform types.

(7) FIG. **5**B is a conceptual diagram illustrating example spatially varying transforms (SVT).

(8) FIG. **6**A is a conceptual diagram illustrating one example of a filter shape used in an adaptive loop filter (ALF).

(9) FIG. **6**B is a conceptual diagram illustrating another example of a filter shape used in an ALF.

(10) FIG. **6**C is a conceptual diagram illustrating another example of a filter shape used in an ALF.

(11) FIG. **7** is a block diagram indicating one example of a specific configuration of a loop filter which functions as a deblocking filter (DBF).

(12) FIG. **8** is a conceptual diagram indicating an example of a deblocking filter having a symmetrical filtering characteristic with respect to a block boundary.

(13) FIG. **9** is a conceptual diagram for illustrating a block boundary on which a deblocking filter process is performed.

(14) FIG. **10** is a conceptual diagram indicating examples of Bs values.

(15) FIG. **11** is a flow chart illustrating one example of a process performed by a prediction processor of the encoder.

(16) FIG. **12** is a flow chart illustrating another example of a process performed by the prediction processor of the encoder.

(17) FIG. **13** is a flow chart illustrating another example of a process performed by the prediction processor of the encoder.
(18) FIG. **14** is a conceptual diagram illustrating sixty-seven intra prediction modes used in intra prediction in an embodiment.
(19) FIG. **15** is a flow chart illustrating an example basic processing flow of inter prediction.
(20) FIG. **16** is a flow chart illustrating one example of derivation of motion vectors.
(21) FIG. **17** is a flow chart illustrating another example of derivation of motion vectors.
(22) FIG. **18** is a flow chart illustrating another example of derivation of motion vectors.
(23) FIG. **19** is a flow chart illustrating an example of inter prediction in normal inter mode.
(24) FIG. **20** is a flow chart illustrating an example of inter prediction in merge mode.
(25) FIG. **21** is a conceptual diagram for illustrating one example of a motion vector derivation process in merge mode.
(26) FIG. **22** is a flow chart illustrating one example of frame rate up conversion (FRUC) process.
(27) FIG. **23** is a conceptual diagram for illustrating one example of pattern matching (bilateral matching) between two blocks along a motion trajectory.
(28) FIG. **24** is a conceptual diagram for illustrating one example of pattern matching (template matching) between a template in a current picture and a block in a reference picture.
(29) FIG. **25**A is a conceptual diagram for illustrating one example of deriving a motion vector of each sub-block based on motion vectors of a plurality of neighboring blocks.
(30) FIG. **25**B is a conceptual diagram for illustrating one example of deriving a motion vector of each sub-block in affine mode in which three control points are used.
(31) FIG. **26**A is a conceptual diagram for illustrating an affine merge mode.
(32) FIG. **26**B is a conceptual diagram for illustrating an affine merge mode in which two control points are used.
(33) FIG. **26**C is a conceptual diagram for illustrating an affine merge mode in which three control points are used.
(34) FIG. **27** is a flow chart illustrating one example of a process in affine merge mode.
(35) FIG. **28**A is a conceptual diagram for illustrating an affine inter mode in which two control points are used.
(36) FIG. **28**B is a conceptual diagram for illustrating an affine inter mode in which three control points are used.
(37) FIG. **29** is a flow chart illustrating one example of a process in affine inter mode.
(38) FIG. **30**A is a conceptual diagram for illustrating an affine inter mode in which a current block has three control points and a neighboring block has two control points.
(39) FIG. **30**B is a conceptual diagram for illustrating an affine inter mode in which a current block has two control points and a neighboring block has three control points.
(40) FIG. **31**A is a flow chart illustrating a merge mode process including decoder motion vector refinement (DMVR).
(41) FIG. **31**B is a conceptual diagram for illustrating one example of a DMVR process.
(42) FIG. **32** is a flow chart illustrating one example of generation of a prediction image.
(43) FIG. **33** is a flow chart illustrating another example of generation of a prediction image.
(44) FIG. **34** is a flow chart illustrating another example of generation of a prediction image.
(45) FIG. **35** is a flow chart illustrating one example of a prediction image correction process performed by an overlapped block motion compensation (OBMC) process.
(46) FIG. **36** is a conceptual diagram for illustrating one example of a prediction image correction process performed by an OBMC process.
(47) FIG. **37**A is a conceptual diagram for illustrating generation of two triangular prediction images.
(48) FIG. **37**B is a conceptual diagram for illustrating examples of a first portion of a first partition and first and second sets of samples that may be used in an OBMC process.
(49) FIG. **37**C is a conceptual diagram for illustrating a first portion of a first partition, which is a portion of the first partition that overlaps with a portion of an adjacent partition.
(50) FIG. **38** is a conceptual diagram for illustrating a model assuming uniform linear motion.
(51) FIG. **39** is a conceptual diagram for illustrating one example of a prediction image generation method using a luminance correction process performed by a local illumination compensation (LIC) process.
(52) FIG. **40** is a block diagram illustrating a mounting example of the encoder.
(53) FIG. **41** is a block diagram illustrating a functional configuration of a decoder according to an embodiment.
(54) FIG. **42** is a flow chart illustrating one example of an overall decoding process performed by the decoder.
(55) FIG. **43** is a flow chart illustrating one example of a process performed by a prediction processor of the decoder.
(56) FIG. **44** is a flow chart illustrating another example of a process performed by the prediction processor of the decoder.
(57) FIG. **45** is a flow chart illustrating an example of inter prediction in normal inter mode in the decoder.
(58) FIG. **46** is a block diagram illustrating a mounting example of the decoder.
(59) FIG. **47**A is a conceptual diagram illustrating one example of reference area in Intra Block Copy (IBC) mode.
(60) FIG. **47**B is a conceptual diagram illustrating one example of a reference block in IBC mode.
(61) FIG. **48** is a flow chart illustrating one example of a method of predicting an image block using IBC mode.
(62) FIG. **49** is a flow chart illustrating another example of a method of predicting an image block using IBC mode.

(63) FIG. **50** is a flow chart illustrating another example of a method of predicting an image block using IBC mode.
(64) FIG. **51**A is a conceptual diagram illustrating one example of a reference block in IBC mode.
(65) FIG. **51**B is a conceptual diagram illustrating another example of a reference block in IBC mode.
(66) FIG. **52**A is a conceptual diagram illustrating another example of a reference block in IBC mode.
(67) FIG. **52**B is a conceptual diagram illustrating another example of a reference block in IBC mode.
(68) FIG. **53** is a block diagram illustrating an overall configuration of a content providing system for implementing a content distribution service.
(69) FIG. **54** is a conceptual diagram illustrating one example of an encoding structure in scalable encoding.
(70) FIG. **55** is a conceptual diagram illustrating one example of an encoding structure in scalable encoding.
(71) FIG. **56** is a conceptual diagram illustrating an example of a display screen of a web page.
(72) FIG. **57** is a conceptual diagram illustrating an example of a display screen of a web page.
(73) FIG. **58** is a block diagram illustrating one example of a smartphone.
(74) FIG. **59** is a block diagram illustrating an example of a configuration of a smartphone.

DETAILED DESCRIPTION

(75) According to one aspect, an image encoder includes circuitry and a memory coupled to the circuitry. The circuitry, in operation, responds to a size of a block satisfying a size condition by generating a prediction image using a prediction mode selected from a plurality of prediction modes. The plurality of prediction modes includes a first prediction mode in which a prediction process uses a motion vector and a reference block in a same picture as the block. The circuitry encodes the block using the prediction image.

(76) According to another aspect, the first prediction mode is intra block copy mode. According to another aspect, the size condition is that both of width and height of the block is less than or equal to a first value. According to another aspect, the first value is 64. According to another aspect, the circuitry codes a flag indicating whether or not performing the first prediction mode. According to another aspect, the circuitry, in response to the size of the block not satisfying the size condition, generates the prediction image using a prediction mode selected from a plurality of prediction modes not including the first prediction mode.

(77) According to another aspect, an image encoder includes: a splitter which, in operation, receives and splits an original picture into blocks; a first adder which, in operation, receives the blocks from the splitter and a prediction image from a predictor, and subtracts each prediction image from its corresponding block to output a residual; a transformer which, in operation, performs a transform on the residuals outputted from the adder to output transform coefficients; a quantizer which, in operation, quantizes the transform coefficients to generate quantized transform coefficients; an entropy encoder which, in operation, encodes the quantized transform coefficients to generate a bitstream; an inverse quantizer and transformer which, in operation, inverse quantizes the quantized transform coefficients to obtain the transform coefficients and inverse transforms the transform coefficients to obtain the residuals; a second adder which, in operation, adds the residuals outputted from the inverse quantizer and transformer and the predictions outputted from the predictor to reconstruct the blocks; and the predictor coupled to a memory, wherein the predictor, in operation, in response to a size of a block satisfying a size condition, generates the prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode, the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block.

(78) According to another aspect, an image decoder is provided including circuitry and a memory coupled to the circuitry. The circuitry, in operation, responds to a size of a block satisfying a size condition by generating a prediction image using a prediction mode selected from a plurality of prediction modes. The plurality of prediction modes includes a first prediction mode, in which a prediction process uses a motion vector and a reference block in a same picture as the block. The circuitry decodes the block using the prediction image.

(79) According to another aspect, the first prediction mode is intra block copy mode. According to another aspect, the size condition is that both of width and height of the block is less than or equal to a first value. According to another aspect, the first value is 64. According to another aspect, the circuitry decodes a flag indicating whether or not performing the first prediction mode. According to another aspect, the circuitry, in response to the size of the block not satisfying the size condition, generates the prediction image using a prediction mode selected from a plurality of prediction modes not including the first prediction mode.

(80) According to another aspect, an image decoder includes: an entropy decoder which, in operation, receives and decodes an encoded bitstream to obtain quantized transform coefficients, an inverse quantizer and transformer which, in operation, inverse quantizes the quantized transform coefficients to obtain transform coefficients and inverse transform the transform coefficients to obtain residuals, an adder which, in operation, adds the residuals outputted from the inverse quantizer and transformer and a prediction image outputted from a predictor to reconstruct blocks, and the predictor coupled to a memory, wherein the predictor, in operation, in response to a size of a block satisfying a size condition, generates the prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode, the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block.

(81) In the drawings, identical reference numbers identify similar elements, unless the context indicates otherwise. The sizes and relative positions of elements in the drawings are not necessarily drawn to scale.

(82) Hereinafter, embodiment(s) will be described with reference to the drawings. Note that the embodiment(s) described below each show a general or specific example. The numerical values, shapes, materials, components, the arrangement and connection of the components, steps, the relation and order of the steps, etc., indicated in the following embodiment(s) are mere examples, and are not intended to limit the scope of the claims.
(83) Embodiments of an encoder and a decoder will be described below. The embodiments are examples of an encoder and a decoder to which the processes and/or configurations presented in the description of aspects of the present disclosure are applicable. The processes and/or configurations can also be implemented in an encoder and a decoder different from those according to the embodiments. For example, regarding the processes and/or configurations as applied to the embodiments, any of the following may be implemented:
(84) (1) Any of the components of the encoder or the decoder according to the embodiments presented in the description of aspects of the present disclosure may be substituted or combined with another component presented anywhere in the description of aspects of the present disclosure.
(85) (2) In the encoder or the decoder according to the embodiments, discretionary changes may be made to functions or processes performed by one or more components of the encoder or the decoder, such as addition, substitution, removal, etc., of the functions or processes. For example, any function or process may be substituted or combined with another function or process presented anywhere in the description of aspects of the present disclosure.
(86) (3) In methods implemented by the encoder or the decoder according to the embodiments, discretionary changes may be made such as addition, substitution, and removal of one or more of the processes included in the method. For example, any process in the method may be substituted or combined with another process presented anywhere in the description of aspects of the present disclosure.
(87) (4) One or more components included in the encoder or the decoder according to embodiments may be combined with a component presented anywhere in the description of aspects of the present disclosure, may be combined with a component including one or more functions presented anywhere in the description of aspects of the present disclosure, and may be combined with a component that implements one or more processes implemented by a component presented in the description of aspects of the present disclosure.
(88) (5) A component including one or more functions of the encoder or the decoder according to the embodiments, or a component that implements one or more processes of the encoder or the decoder according to the embodiments, may be combined or substituted with a component presented anywhere in the description of aspects of the present disclosure, with a component including one or more functions presented anywhere in the description of aspects of the present disclosure, or with a component that implements one or more processes presented anywhere in the description of aspects of the present disclosure.
(89) (6) In methods implemented by the encoder or the decoder according to the embodiments, any of the processes included in the method may be substituted or combined with a process presented anywhere in the description of aspects of the present disclosure or with any corresponding or equivalent process.
(90) (7) One or more processes included in methods implemented by the encoder or the decoder according to the embodiments may be combined with a process presented anywhere in the description of aspects of the present disclosure.
(91) (8) The implementation of the processes and/or configurations presented in the description of aspects of the present disclosure is not limited to the encoder or the decoder according to the embodiments. For example, the processes and/or configurations may be implemented in a device used for a purpose different from the moving picture encoder or the moving picture decoder disclosed in the embodiments.
(92) [Encoder]
(93) First, an encoder according to an embodiment will be described. FIG. **1** is a block diagram illustrating a functional configuration of encoder **100** according to the embodiment. Encoder **100** is a video encoder which encodes a video in units of a block.
(94) As illustrated in FIG. **1**, encoder **100** is an apparatus which encodes an image in units of a block, and includes splitter **102**, subtractor **104**, transformer **106**, quantizer **108**, entropy encoder **110**, inverse quantizer **112**, inverse transformer **114**, adder **116**, block memory **118**, loop filter **120**, frame memory **122**, intra predictor **124**, inter predictor **126**, and prediction controller **128**.
(95) Encoder **100** is implemented as, for example, a generic processor and memory. In this case, when a software program stored in the memory is executed by the processor, the processor functions as splitter **102**, subtractor **104**, transformer **106**, quantizer **108**, entropy encoder **110**, inverse quantizer **112**, inverse transformer **114**, adder **116**, loop filter **120**, intra predictor **124**, inter predictor **126**, and prediction controller **128**. Alternatively, encoder **100** may be implemented as one or more dedicated electronic circuits corresponding to splitter **102**, subtractor **104**, transformer **106**, quantizer **108**, entropy encoder **110**, inverse quantizer **112**, inverse transformer **114**, adder **116**, loop filter **120**, intra predictor **124**, inter predictor **126**, and prediction controller **128**.
(96) Hereinafter, an overall flow of processes performed by encoder **100** is described, and then each of constituent elements included in encoder **100** will be described.
(97) [Overall Flow of Encoding Process]
(98) FIG. **2** is a flow chart indicating one example of an overall encoding process performed by encoder **100**.

(99) First, splitter **102** of encoder **100** splits each of pictures included in an input image, which is a video, into a plurality of blocks having a fixed size (e.g., 128×128 pixels) (Step Sa_**1**). Splitter **102** then selects a splitting pattern for the fixed-size block (also referred to as a block shape) (Step Sa_**2**). In other words, splitter **102** further splits the fixed-size block into a plurality of blocks which form the selected splitting pattern. Encoder **100** performs, for each of the plurality of blocks, Steps Sa_**3** to Sa_**9** for the block (that is a current block to be encoded).
(100) In other words, a prediction processor which includes all or part of intra predictor **124**, inter predictor **126**, and prediction controller **128** generates a prediction signal (also referred to as a prediction block) of the current block to be encoded (also referred to as a current block) (Step Sa_**3**).
(101) Next, subtractor **104** generates a difference between the current block and a prediction block as a prediction residual (also referred to as a difference block) (Step Sa_**4**).
(102) Next, transformer **106** transforms the difference block and quantizer **108** quantizes the result, to generate a plurality of quantized coefficients (Step Sa_**5**). It is to be noted that the block having the plurality of quantized coefficients is also referred to as a coefficient block.
(103) Next, entropy encoder **110** encodes (specifically, entropy encodes) the coefficient block and a prediction parameter related to generation of a prediction signal, to generate an encoded signal (Step Sa_**6**). It is to be noted that the encoded signal is also referred to as an encoded bitstream, a compressed bitstream, or a stream.
(104) Next, inverse quantizer **112** performs inverse quantization of the coefficient block and inverse transformer **114** performs inverse transform of the result, to restore a plurality of prediction residuals (that is, a difference block) (Step Sa_**7**).
(105) Next, adder **116** adds the prediction block to the restored difference block to reconstruct the current block as a reconstructed image (also referred to as a reconstructed block or a decoded image block) (Step Sa_**8**). In this way, the reconstructed image is generated.
(106) When the reconstructed image is generated, loop filter **120** performs filtering of the reconstructed image as necessary (Step Sa_**9**).
(107) Encoder **100** then determines whether encoding of the entire picture has been finished (Step Sa_**10**). When determining that the encoding has not yet been finished (No in Step Sa_**10**), processes from Step Sa_**2** are executed repeatedly.
(108) Although encoder **100** selects one splitting pattern for a fixed-size block, and encodes each block according to the splitting pattern in the above-described example, it is to be noted that each block may be encoded according to a corresponding one of a plurality of splitting patterns. In this case, encoder **100** may evaluate a cost for each of the plurality of splitting patterns, and, for example, may select the encoded signal obtainable by encoding according to the splitting pattern which yields the smallest cost as an encoded signal which is output.
(109) As illustrated, the processes in Steps Sa_**1** to Sa_**10** are performed sequentially by encoder **100**. Alternatively, two or more of the processes may be performed in parallel, the processes may be reordered, etc.
(110) [Splitter]
(111) Splitter **102** splits each of pictures included in an input video into a plurality of blocks, and outputs each block to subtractor **104**. For example, splitter **102** first splits a picture into blocks of a fixed size (for example, 128×128). Other fixed block sizes may be employed. The fixed-size block is also referred to as a coding tree unit (CTU). Splitter **102** then splits each fixed-size block into blocks of variable sizes (for example, 64×64 or smaller), based on recursive quadtree and/or binary tree block splitting. In other words, splitter **102** selects a splitting pattern. The variable-size block is also referred to as a coding unit (CU), a prediction unit (PU), or a transform unit (TU). It is to be noted that, in various kinds of processing examples, there is no need to differentiate between CU, PU, and TU; all or some of the blocks in a picture may be processed in units of a CU, a PU, or a TU.
(112) FIG. **3** is a conceptual diagram illustrating one example of block splitting according to an embodiment. In FIG. **3**, the solid lines represent block boundaries of blocks split by quadtree block splitting, and the dashed lines represent block boundaries of blocks split by binary tree block splitting.
(113) Here, block **10** is a square block having 128×128 pixels (128×128 block). This 128×128 block **10** is first split into four square 64×64 blocks (quadtree block splitting).
(114) The upper-left 64×64 block is further vertically split into two rectangular 32×64 blocks, and the left 32×64 block is further vertically split into two rectangular 16×64 blocks (binary tree block splitting). As a result, the upper-left 64×64 block is split into two 16×64 blocks **11** and **12** and one 32×64 block **13**.
(115) The upper-right 64×64 block is horizontally split into two rectangular 64×32 blocks **14** and **15** (binary tree block splitting).
(116) The lower-left 64×64 block is first split into four square 32×32 blocks (quadtree block splitting). The upper-left block and the lower-right block among the four 32×32 blocks are further split. The upper-left 32×32 block is vertically split into two rectangle 16×32 blocks, and the right 16×32 block is further horizontally split into two 16×16 blocks (binary tree block splitting). The lower-right 32×32 block is horizontally split into two 32×16 blocks (binary tree block splitting). As a result, the lower-left 64×64 block is split into 16×32 block **16**, two 16×16 blocks **17** and **18**, two 32×32 blocks **19** and **20**, and two 32×16 blocks **21** and **22**.
(117) The lower-right 64×64 block **23** is not split.

(118) As described above, in FIG. **3**, block **10** is split into thirteen variable-size blocks **11** through **23** based on recursive quadtree and binary tree block splitting. This type of splitting is also referred to as quadtree plus binary tree (QTBT) splitting.
(119) It is to be noted that, in FIG. **3**, one block is split into four or two blocks (quadtree or binary tree block splitting), but splitting is not limited to these examples. For example, one block may be split into three blocks (ternary block splitting). Splitting including such ternary block splitting is also referred to as multi-type tree (MBT) splitting.
(120) [Picture Structure: Slice/Tile]
(121) A picture may be configured in units of one or more slices or tiles in order to decode the picture in parallel. The picture configured in units of one or more slices or tiles may be configured by splitter **102**.
(122) Slices are basic encoding units included in a picture. A picture may include, for example, one or more slices. In addition, a slice includes one or more successive coding tree units (CTU).
(123) FIG. **4**A is a conceptual diagram illustrating one example of a slice configuration. For example, a picture includes 11×8 CTUs and is split into four slices (slices **1** to **4**). Slice **1** includes sixteen CTUs, slice **2** includes twenty-one CTUs, slice **3** includes twenty-nine CTUs, and slice **4** includes twenty-two CTUs. Here, each CTU in the picture belongs to one of the slices. The shape of each slice is a shape obtainable by splitting the picture horizontally. A boundary of each slice does not need to be coincide with an image end, and may coincide with any of the boundaries between CTUs in the image. The processing order of the CTUs in a slice (an encoding order or a decoding order) is, for example, a raster-scan order. A slice includes header information and encoded data. Features of the slice may be described in header information. The features include a CTU address of a top CTU in the slice, a slice type, etc.
(124) A tile is a unit of a rectangular region included in a picture. Each tile may be assigned with a number referred to as TileId in raster-scan order.
(125) FIG. **4**B is a conceptual diagram indicating an example of a tile configuration. For example, a picture includes 11×8 CTUs and is split into four tiles of rectangular regions (tiles **1** to **4**). When tiles are used, the processing order of CTUs are changed from the processing order in the case where no tile is used. When no tile is used, CTUs in a picture are processed in raster-scan order. When tiles are used, at least one CTU in each of the tiles is processed in raster-scan order. For example, as illustrated in FIG. **4**B, the processing order of the CTUs included in tile **1** is the order which starts from the left-end of the first row of tile **1** toward the right-end of the first row of tile **1** and then starts from the left-end of the second row of tile **1** toward the right-end of the second row of tile **1**.
(126) It is to be noted that the one tile may include one or more slices, and one slice may include one or more tiles.
(127) [Subtractor]
(128) Subtractor **104** subtracts a prediction signal (prediction sample that is input from prediction controller **128** indicated below) from an original signal (original sample) in units of a block input from splitter **102** and split by splitter **102**. In other words, subtractor **104** calculates prediction errors (also referred to as residuals) of a block to be encoded (hereinafter also referred to as a current block). Subtractor **104** then outputs the calculated prediction errors (residuals) to transformer **106**.
(129) The original signal is a signal which has been input into encoder **100** and represents an image of each picture included in a video (for example, a luma signal and two chroma signals). Hereinafter, a signal representing an image is also referred to as a sample.
(130) [Transformer]
(131) Transformer **106** transforms prediction errors in spatial domain into transform coefficients in frequency domain, and outputs the transform coefficients to quantizer **108**. More specifically, transformer **106** applies, for example, a defined discrete cosine transform (DCT) or discrete sine transform (DST) to prediction errors in spatial domain. The defined DCT or DST may be predefined.
(132) It is to be noted that transformer **106** may adaptively select a transform type from among a plurality of transform types, and transform prediction errors into transform coefficients by using a transform basis function corresponding to the selected transform type. This sort of transform is also referred to as explicit multiple core transform (EMT) or adaptive multiple transform (AMT).
(133) The transform types include, for example, DCT-II, DCT-V, DCT-VIII, DST-I, and DST-VII. FIG. **5**A is a chart indicating transform basis functions for the example transform types. In FIG. **5**A, N indicates the number of input pixels. For example, selection of a transform type from among the plurality of transform types may depend on a prediction type (one of intra prediction and inter prediction), and may depend on an intra prediction mode.
(134) Information indicating whether to apply such EMT or AMT (referred to as, for example, an EMT flag or an AMT flag) and information indicating the selected transform type is normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the bit sequence level, picture level, slice level, tile level, or CTU level).
(135) In addition, transformer **106** may re-transform the transform coefficients (transform result). Such re-transform is also referred to as adaptive secondary transform (AST) or non-separable secondary transform (NSST). For example, transformer **106** performs re-transform in units of a sub-block (for example, 4×4 sub-block) included in a transform coefficient block corresponding to an intra prediction error. Information indicating whether to apply NSST and information related to a transform matrix for use in NSST are normally signaled at the CU level. It is to be noted that

the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, or CTU level).
(136) Transformer **106** may employ a separable transform and a non-separable transform. A separable transform is a method in which a transform is performed a plurality of times by separately performing a transform for each of a number of directions according to the number of dimensions of inputs. A non-separable transform is a method of performing a collective transform in which two or more dimensions in multidimensional inputs are collectively regarded as a single dimension.
(137) In one example of a non-separable transform, when an input is a 4×4 block, the 4×4 block is regarded as a single array including sixteen elements, and the transform applies a 16×16 transform matrix to the array.
(138) In another example of a non-separable transform, a 4×4 input block is regarded as a single array including sixteen elements, and then a transform (hypercube givens transform) in which givens revolution is performed on the array a plurality of times may be performed.
(139) In the transform in transformer **106**, the types of bases to be transformed into the frequency domain according to regions in a CU can be switched. Examples include spatially varying transforms (SVT). In SVT, as illustrated in FIG. **5**B, CUs are split into two equal regions horizontally or vertically, and only one of the regions is transformed into the frequency domain. A transform basis type can be set for each region. For example, DST7 and DST8 are used. In this example, only one of these two regions in the CU is transformed, and the other is not transformed. However, both of these two regions may be transformed. In addition, the splitting method is not limited to the splitting into two equal regions, and can be more flexible. For example, the CU may be split into four equal regions, or information indicating splitting may be encoded separately and be signaled in the same manner as the CU splitting. It is to be noted that SVT is also referred to as sub-block transform (SBT).
(140) [Quantizer]
(141) Quantizer **108** quantizes the transform coefficients output from transformer **106**. More specifically, quantizer **108** scans, in a determined scanning order, the transform coefficients of the current block, and quantizes the scanned transform coefficients based on quantization parameters (QP) corresponding to the transform coefficients. Quantizer **108** then outputs the quantized transform coefficients (hereinafter also referred to as quantized coefficients) of the current block to entropy encoder **110** and inverse quantizer **112**. The determined scanning order may be predetermined.
(142) A determined scanning order is an order for quantizing/inverse quantizing transform coefficients. For example, a determined scanning order may be defined as ascending order of frequency (from low to high frequency) or descending order of frequency (from high to low frequency).
(143) A quantization parameter (QP) is a parameter defining a quantization step (quantization width). For example, when the value of the quantization parameter increases, the quantization step also increases. In other words, when the value of the quantization parameter increases, the quantization error increases.
(144) In addition, a quantization matrix may be used for quantization. For example, several kinds of quantization matrices may be used correspondingly to frequency transform sizes such as 4×4 and 8×8, prediction modes such as intra prediction and inter prediction, and pixel components such as luma and chroma pixel components. It is to be noted that quantization means digitalizing values sampled at determined intervals correspondingly to determined levels. In this technical field, quantization may be referred to using other expressions, such as rounding and scaling, and may employ rounding and scaling. The determined intervals and levels may be predetermined.
(145) Methods using quantization matrices include a method using a quantization matrix which has been set directly at the encoder side, and a method using a quantization matrix which has been set as a default (default matrix). At the encoder side, a quantization matrix suitable for features of an image can be set by directly setting a quantization matrix. This case, however, has a disadvantage of increasing a coding amount for encoding the quantization matrix.
(146) There is a method for quantizing a high-frequency coefficient and a low-frequency coefficient without using a quantization matrix. It is to be noted that this method is equivalent to a method using a quantization matrix (flat matrix) whose coefficients have the same value.
(147) The quantization matrix may be specified using, for example, a sequence parameter set (SPS) or a picture parameter set (PPS). The SPS includes a parameter which is used for a sequence, and the PPS includes a parameter which is used for a picture. Each of the SPS and the PPS may be simply referred to as a parameter set.
(148) [Entropy Encoder]
(149) Entropy encoder **110** generates an encoded signal (encoded bitstream) based on quantized coefficients which have been input from quantizer **108**. More specifically, entropy encoder **110**, for example, binarizes quantized coefficients, arithmetically encodes the binary signal, and outputs a compressed bit stream or sequence.
(150) [Inverse Quantizer]
(151) Inverse quantizer **112** inverse quantizes quantized coefficients which have been input from quantizer **108**. More specifically, inverse quantizer **112** inverse quantizes, in a determined scanning order, quantized coefficients of the current block. Inverse quantizer **112** then outputs the inverse quantized transform coefficients of the current block to inverse transformer **114**. The determined scanning order may be predetermined.
(152) [Inverse Transformer]
(153) Inverse transformer **114** restores prediction errors (residuals) by inverse transforming transform coefficients

which have been input from inverse quantizer **112**. More specifically, inverse transformer **114** restores the prediction errors of the current block by applying an inverse transform corresponding to the transform applied by transformer **106** on the transform coefficients. Inverse transformer **114** then outputs the restored prediction errors to adder **116**.
(154) It is to be noted that since information is lost in quantization, the restored prediction errors do not match the prediction errors calculated by subtractor **104**. In other words, the restored prediction errors normally include quantization errors.
(155) [Adder]
(156) Adder **116** reconstructs the current block by adding prediction errors which have been input from inverse transformer **114** and prediction samples which have been input from prediction controller **128**. Adder **116** then outputs the reconstructed block to block memory **118** and loop filter **120**. A reconstructed block is also referred to as a local decoded block.
(157) [Block Memory]
(158) Block memory **118** is, for example, storage for storing blocks in a picture to be encoded (hereinafter referred to as a current picture) which is referred to in intra prediction. More specifically, block memory **118** stores reconstructed blocks output from adder **116**.
(159) [Frame Memory]
(160) Frame memory **122** is, for example, storage for storing reference pictures for use in inter prediction, and is also referred to as a frame buffer. More specifically, frame memory **122** stores reconstructed blocks filtered by loop filter **120**.
(161) [Loop Filter]
(162) Loop filter **120** applies a loop filter to blocks reconstructed by adder **116**, and outputs the filtered reconstructed blocks to frame memory **122**. A loop filter is a filter used in an encoding loop (in-loop filter), and includes, for example, a deblocking filter (DF or DBF), a sample adaptive offset (SAO), and an adaptive loop filter (ALF).
(163) In an ALF, a least square error filter for removing compression artifacts is applied. For example, one filter selected from among a plurality of filters based on the direction and activity of local gradients is applied for each of 2×2 sub-blocks in the current block.
(164) More specifically, first, each sub-block (for example, each 2×2 sub-block) is categorized into one out of a plurality of classes (for example, fifteen or twenty-five classes). The classification of the sub-block is based on gradient directionality and activity. For example, classification index C (for example, C=5D+A) is derived based on gradient directionality D (for example, 0 to 2 or 0 to 4) and gradient activity A (for example, 0 to 4). Then, based on classification index C, each sub-block is categorized into one out of a plurality of classes.
(165) For example, gradient directionality D is calculated by comparing gradients of a plurality of directions (for example, the horizontal, vertical, and two diagonal directions). Moreover, for example, gradient activity A is calculated by adding gradients of a plurality of directions and quantizing the result of addition.
(166) The filter to be used for each sub-block is determined from among the plurality of filters based on the result of such categorization.
(167) The filter shape to be used in an ALF is, for example, a circular symmetric filter shape. FIG. **6**A through FIG. **6**C illustrate examples of filter shapes used in ALFs. FIG. **6**A illustrates a 5×5 diamond shape filter, FIG. **6**B illustrates a 7×7 diamond shape filter, and FIG. **6**C illustrates a 9×9 diamond shape filter. Information indicating the filter shape is normally signaled at the picture level. It is to be noted that the signaling of such information indicating the filter shape does not necessarily need to be performed at the picture level, and may be performed at another level (for example, at the sequence level, slice level, tile level, CTU level, or CU level).
(168) The ON or OFF of the ALF is determined, for example, at the picture level or CU level. For example, the decision of whether to apply the ALF to luma may be made at the CU level, and the decision of whether to apply ALF to chroma may be made at the picture level. Information indicating ON or OFF of the ALF is normally signaled at the picture level or CU level. It is to be noted that the signaling of information indicating ON or OFF of the ALF does not necessarily need to be performed at the picture level or CU level, and may be performed at another level (for example, at the sequence level, slice level, tile level, or CTU level).
(169) The coefficient set for the plurality of selectable filters (for example, fifteen or up to twenty-five filters) is normally signaled at the picture level. It is to be noted that the signaling of the coefficient set does not necessarily need to be performed at the picture level, and may be performed at another level (for example, at the sequence level, slice level, tile level, CTU level, CU level, or sub-block level).
(170) [Loop Filter>Deblocking Filter]
(171) In a deblocking filter, loop filter **120** performs a filter process on a block boundary in a reconstructed image so as to reduce distortion which occurs at the block boundary.
(172) FIG. **7** is a block diagram illustrating one example of a specific configuration of loop filter **120** which functions as a deblocking filter.
(173) Loop filter **120** includes: boundary determiner **1201**; filter determiner **1203**; filtering executor **1205**; process determiner **1208**; filter characteristic determiner **1207**; and switches **1202**, **1204**, and **1206**.
(174) Boundary determiner **1201** determines whether a pixel to be deblock-filtered (that is, a current pixel) is present

around a block boundary. Boundary determiner **1201** then outputs the determination result to switch **1202** and processing determiner **1208**.
(175) In the case where boundary determiner **1201** has determined that a current pixel is present around a block boundary, switch **1202** outputs an unfiltered image to switch **1204**. In the opposite case where boundary determiner **1201** has determined that no current pixel is present around a block boundary, switch **1202** outputs an unfiltered image to switch **1206**.
(176) Filter determiner **1203** determines whether to perform deblocking filtering of the current pixel, based on the pixel value of at least one surrounding pixel located around the current pixel. Filter determiner **1203** then outputs the determination result to switch **1204** and processing determiner **1208**.
(177) In the case where filter determiner **1203** has determined to perform deblocking filtering of the current pixel, switch **1204** outputs the unfiltered image obtained through switch **1202** to filtering executor **1205**. In the opposite case where filter determiner **1203** has determined not to perform deblocking filtering of the current pixel, switch **1204** outputs the unfiltered image obtained through switch **1202** to switch **1206**.
(178) When obtaining the unfiltered image through switches **1202** and **1204**, filtering executor **1205** executes, for the current pixel, deblocking filtering with the filter characteristic determined by filter characteristic determiner **1207**. Filtering executor **1205** then outputs the filtered pixel to switch **1206**.
(179) Under control by processing determiner **1208**, switch **1206** selectively outputs a pixel which has not been deblock-filtered and a pixel which has been deblock-filtered by filtering executor **1205**.
(180) Processing determiner **1208** controls switch **1206** based on the results of determinations made by boundary determiner **1201** and filter determiner **1203**. In other words, processing determiner **1208** causes switch **1206** to output the pixel which has been deblock-filtered when boundary determiner **1201** has determined that the current pixel is present around the block boundary and when filter determiner **1203** has determined to perform deblocking filtering of the current pixel. In addition, other than the above case, processing determiner **1208** causes switch **1206** to output the pixel which has not been deblock-filtered. A filtered image is output from switch **1206** by repeating output of a pixel in this way.
(181) FIG. **8** is a conceptual diagram indicating an example of a deblocking filter having a symmetrical filtering characteristic with respect to a block boundary.
(182) In a deblocking filter process, one of two deblocking filters having different characteristics, that is, a strong filter and a weak filter, is selected using pixel values and quantization parameters. In the case of the strong filter, when pixels p**0** to p**2** and pixels q**0** to q**2** are present across a block boundary as illustrated in FIG. **8**, the pixel values of the respective pixel q**0** to q**2** are changed to pixel values q′**0** to q′**2** by performing, for example, computations according to the expressions below.
(183)
$q'0=(p1+2\times p0+2\times q0+2\times q1+q2+4)/8q'1=(p0+q0+q1+q2+2)/4q'2=(p+q0+q1+3\times q2+2\times q3+4)/$
(184) It is to be noted that, in the above expressions, p**0** to p**2** and q**0** to q**2** are the pixel values of respective pixels p**0** to p**2** and pixels q**0** to q**2**. In addition, q**3** is the pixel value of neighboring pixel q**3** located at the opposite side of pixel q**2** with respect to the block boundary. In addition, in the right side of each of the expressions, coefficients which are multiplied with the respective pixel values of the pixels to be used for deblocking filtering are filter coefficients.
(185) Furthermore, in the deblocking filtering, clipping may be performed so that the calculated pixel values are not set over a threshold value. In the clipping process, the pixel values calculated according to the above expressions are clipped to a value obtained according to "a computation pixel value±2×a threshold value" using the threshold value determined based on a quantization parameter. In this way, it is possible to prevent excessive smoothing.
(186) FIG. **9** is a conceptual diagram for illustrating a block boundary on which a deblocking filter process is performed. FIG. **10** is a conceptual diagram indicating examples of Bs values.
(187) The block boundary on which the deblocking filter process is performed is, for example, a boundary between prediction units (PU) having 8×8 pixel blocks as illustrated in FIG. **9**, or a boundary between transform units (TU). The deblocking filter process may be performed in units of four rows or four columns. First, boundary strength (Bs) values are determined as indicated in FIG. **10** for block P and block Q illustrated in FIG. **9**.
(188) According to the Bs values in FIG. **10**, whether to perform deblocking filter processes of block boundaries belonging to the same image using different strengths is determined. The deblocking filter process for a chroma signal is performed when a Bs value is 2. The deblocking filter process for a luma signal is performed when a Bs value is 1 or more and a determined condition is satisfied. The determined condition may be predetermined. It is to be noted that conditions for determining Bs values are not limited to those indicated in FIG. **10**, and a Bs value may be determined based on another parameter.
(189) [Prediction Processor (Intra Predictor, Inter Predictor, Prediction Controller)]
(190) FIG. **11** is a flow chart illustrating one example of a process performed by the prediction processor of encoder **100**. It is to be noted that the prediction processor includes all or part of the following constituent elements: intra predictor **124**; inter predictor **126**; and prediction controller **128**.
(191) The prediction processor generates a prediction image of a current block (Step Sb_**1**). This prediction image is also referred to as a prediction signal or a prediction block. It is to be noted that the prediction signal is, for example, an

intra prediction signal or an inter prediction signal. Specifically, the prediction processor generates the prediction image of the current block using a reconstructed image which has been already obtained through generation of a prediction block, generation of a difference block, generation of a coefficient block, restoring of a difference block, and generation of a decoded image block.
(192) The reconstructed image may be, for example, an image in a reference picture, or an image of an encoded block in a current picture which is the picture including the current block. The encoded block in the current picture is, for example, a neighboring block of the current block.
(193) FIG. **12** is a flow chart illustrating another example of a process performed by the prediction processor of encoder **100**.
(194) The prediction processor generates a prediction image using a first method (Step Sc_**1***a*), generates a prediction image using a second method (Step Sc_**1***b*), and generates a prediction image using a third method (Step Sc_**1***c*). The first method, the second method, and the third method may be mutually different methods for generating a prediction image. Each of the first to third methods may be an inter prediction method, an intra prediction method, or another prediction method. The above-described reconstructed image may be used in these prediction methods.
(195) Next, the prediction processor selects any one of a plurality of prediction methods generated in Steps Sc_**1***a*, Sc_**1***b*, and Sc_**1***c* (Step Sc_**2**). The selection of the prediction image, that is selection of a method or a mode for obtaining a final prediction image, may be made by calculating a cost for each of the generated prediction images and be based on the cost. Alternatively, the selection of the prediction image may be made based on a parameter which is used in an encoding process. Encoder **100** may transform information for identifying a selected prediction image, a method, or a mode into an encoded signal (also referred to as an encoded bitstream). The information may be, for example, a flag or the like. In this way, the decoder is capable of generating a prediction image according to the method or the mode selected based on the information in encoder **100**. It is to be noted that, in the example illustrated in FIG. **12**, the prediction processor selects any of the prediction images after the prediction images are generated using the respective methods. However, the prediction processor may select a method or a mode based on a parameter for use in the above-described encoding process before generating prediction images, and may generate a prediction image according to the method or mode selected.
(196) For example, the first method and the second method may be intra prediction and inter prediction, respectively, and the prediction processor may select a final prediction image for a current block from prediction images generated according to the prediction methods.
(197) FIG. **13** is a flow chart illustrating another example of a process performed by the prediction processor of encoder **100**.
(198) First, the prediction processor generates a prediction image using intra prediction (Step Sd_**1***a*), and generates a prediction image using inter prediction (Step Sd_**1***b*). It is to be noted that the prediction image generated by intra prediction is also referred to as an intra prediction image, and the prediction image generated by inter prediction is also referred to as an inter prediction image.
(199) Next, the prediction processor evaluates each of the intra prediction image and the inter prediction image (Step Sd_**2**). A cost may be used in the evaluation. In other words, the prediction processor calculates cost C for each of the intra prediction image and the inter prediction image. Cost C may be calculated according to an expression of an R-D optimization model, for example, $C=D+\lambda\times R$. In this expression, D indicates a coding distortion of a prediction image, and is represented as, for example, a sum of absolute differences between the pixel value of a current block and the pixel value of a prediction image. In addition, R indicates a predicted coding amount of a prediction image, specifically, the coding amount required to encode motion information for generating a prediction image, etc. In addition, λ indicates, for example, a multiplier according to the method of Lagrange multiplier.
(200) The prediction processor then selects the prediction image for which the smallest cost C has been calculated among the intra prediction image and the inter prediction image, as the final prediction image for the current block (Step Sd_**3**). In other words, the prediction method or the mode for generating the prediction image for the current block is selected.
(201) [Intra Predictor]
(202) Intra predictor **124** generates a prediction signal (intra prediction signal) by performing intra prediction (also referred to as intra frame prediction) of the current block by referring to a block or blocks in the current picture and stored in block memory **118**. More specifically, intra predictor **124** generates an intra prediction signal by performing intra prediction by referring to samples (for example, luma and/or chroma values) of a block or blocks neighboring the current block, and then outputs the intra prediction signal to prediction controller **128**.
(203) For example, intra predictor **124** performs intra prediction by using one mode from among a plurality of intra prediction modes which have been defined. The intra prediction modes include one or more non-directional prediction modes and a plurality of directional prediction modes. The defined modes may be predefined.
(204) The one or more non-directional prediction modes include, for example, the planar prediction mode and DC prediction mode defined in the H.265/high-efficiency video coding (HEVC) standard.
(205) The plurality of directional prediction modes include, for example, the thirty-three directional prediction modes defined in the H.265/HEVC standard. It is to be noted that the plurality of directional prediction modes may further

include thirty-two directional prediction modes in addition to the thirty-three directional prediction modes (for a total of sixty-five directional prediction modes). FIG. **14** is a conceptual diagram illustrating sixty-seven intra prediction modes in total that may be used in intra prediction (two non-directional prediction modes and sixty-five directional prediction modes). The solid arrows represent the thirty-three directions defined in the H.265/HEVC standard, and the dashed arrows represent the additional thirty-two directions (the two non-directional prediction modes are not illustrated in FIG. **14**).
(206) In various kinds of processing examples, a luma block may be referred to in intra prediction of a chroma block. In other words, a chroma component of the current block may be predicted based on a luma component of the current block. Such intra prediction is also referred to as cross-component linear model (CCLM) prediction. The intra prediction mode for a chroma block in which such a luma block is referred to (also referred to as, for example, a CCLM mode) may be added as one of the intra prediction modes for chroma blocks.
(207) Intra predictor **124** may correct intra-predicted pixel values based on horizontal/vertical reference pixel gradients. Intra prediction accompanied by this sort of correcting is also referred to as position dependent intra prediction combination (PDPC). Information indicating whether to apply PDPC (referred to as, for example, a PDPC flag) is normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, or CTU level).
(208) [Inter Predictor]
(209) Inter predictor **126** generates a prediction signal (inter prediction signal) by performing inter prediction (also referred to as inter frame prediction) of the current block by referring to a block or blocks in a reference picture, which is different from the current picture and is stored in frame memory **122**. Inter prediction is performed in units of a current block or a current sub-block (for example, a 4×4 block) in the current block. For example, inter predictor **126** performs motion estimation in a reference picture for the current block or the current sub-block, and finds out a reference block or a sub-block which best matches the current block or the current sub-block. Inter predictor **126** then obtains motion information (for example, a motion vector) which compensates a motion or a change from the reference block or the sub-block to the current block or the sub-block. Inter predictor **126** generates an inter prediction signal of the current block or the sub-block by performing motion compensation (or motion prediction) based on the motion information. Inter predictor **126** outputs the generated inter prediction signal to prediction controller **128**.
(210) The motion information used in motion compensation may be signaled as inter prediction signals in various forms. For example, a motion vector may be signaled. As another example, the difference between a motion vector and a motion vector predictor may be signaled.
(211) [Basic Flow of Inter Prediction]
(212) FIG. **15** is a flow chart illustrating an example basic processing flow of inter prediction.
(213) First, inter predictor **126** generates a prediction signal (Steps Se_**1** to Se_**3**). Next, subtractor **104** generates the difference between a current block and a prediction image as a prediction residual (Step Se_**4**).
(214) Here, in the generation of the prediction image, inter predictor **126** generates the prediction image through determination of a motion vector (MV) of the current block (Steps Se_**1** and Se_**2**) and motion compensation (Step Se_**3**). Furthermore, in determination of an MV, inter predictor **126** determines the MV through selection of a motion vector candidate (MV candidate) (Step Se_**1**) and derivation of an MV (Step Se_**2**). The selection of the MV candidate is made by, for example, selecting at least one MV candidate from an MV candidate list. Alternatively, in derivation of an MV, inter predictor **126** may further select at least one MV candidate from the at least one MV candidate, and determine the selected at least one MV candidate as the MV for the current block. Alternatively, inter predictor **126** may determine the MV for the current block by performing estimation in a reference picture region specified by each of the selected at least one MV candidate. It is to be noted that the estimation in a reference picture region may be referred to as motion estimation.
(215) In addition, although Steps Se_**1** to Se_**3** are performed by inter predictor **126** in the above-described example, a process that is for example Step Se_**1**, Step Se_**2**, or the like may be performed by another constituent element included in encoder **100**.
(216) [Motion Vector Derivation Flow]
(217) FIG. **16** is a flow chart illustrating one example of derivation of motion vectors.
(218) Inter predictor **126** derives an MV of a current block in a mode for encoding motion information (for example, an MV). In this case, for example, the motion information is encoded as a prediction parameter, and is signaled. In other words, the encoded motion information is included in an encoded signal (also referred to as an encoded bitstream).
(219) Alternatively, inter predictor **126** derives an MV in a mode in which motion information is not encoded. In this case, no motion information is included in an encoded signal.
(220) Here, MV derivation modes may include a normal inter mode, a merge mode, a FRUC mode, an affine mode, etc. which are described later. Modes in which motion information is encoded among the modes include the normal inter mode, the merge mode, the affine mode (specifically, an affine inter mode and an affine merge mode), etc. It is to be noted that motion information may include not only an MV but also motion vector predictor selection information which is described later. Modes in which no motion information is encoded include the FRUC mode, etc. Inter predictor

126 selects a mode for deriving an MV of the current block from the modes, and derives the MV of the current block using the selected mode.
(221) FIG. **17** is a flow chart illustrating another example of derivation of motion vectors.
(222) Inter predictor **126** derives an MV of a current block in a mode in which an MV difference is encoded. In this case, for example, the MV difference is encoded as a prediction parameter, and is signaled. In other words, the encoded MV difference is included in an encoded signal. The MV difference is the difference between the MV of the current block and the MV predictor.
(223) Alternatively, inter predictor **126** derives an MV in a mode in which no MV difference is encoded. In this case, no encoded MV difference is included in an encoded signal.
(224) Here, as described above, the MV derivation modes include the normal inter mode, the merge mode, the FRUC mode, the affine mode, etc. which are described later. Modes in which an MV difference is encoded among the modes include the normal inter mode, the affine mode (specifically, the affine inter mode), etc. Modes in which no MV difference is encoded include the FRUC mode, the merge mode, the affine mode (specifically, the affine merge mode), etc. Inter predictor **126** selects a mode for deriving an MV of the current block from the plurality of modes, and derives the MV of the current block using the selected mode.
(225) [Motion Vector Derivation Flow]
(226) FIG. **18** is a flow chart illustrating another example of derivation of motion vectors. The MV derivation modes which are inter prediction modes include a plurality of modes and are roughly divided into modes in which an MV difference is encoded and modes in which no motion vector difference is encoded. The modes in which no MV difference is encoded include the merge mode, the FRUC mode, the affine mode (specifically, the affine merge mode), etc. These modes are described in detail later. Simply, the merge mode is a mode for deriving an MV of a current block by selecting a motion vector from an encoded surrounding block, and the FRUC mode is a mode for deriving an MV of a current block by performing estimation between encoded regions. The affine mode is a mode for deriving, as an MV of a current block, a motion vector of each of a plurality of sub-blocks included in the current block, assuming affine transform.
(227) More specifically, as illustrated when the inter prediction mode information indicates 0 (0 in Sf_**1**), inter predictor **126** derives a motion vector using the merge mode (Sf_**2**). When the inter prediction mode information indicates 1 (1 in Sf_**1**), inter predictor **126** derives a motion vector using the FRUC mode (Sf_**3**). When the inter prediction mode information indicates 2 (2 in Sf_**1**), inter predictor **126** derives a motion vector using the affine mode (specifically, the affine merge mode) (Sf_**4**). When the inter prediction mode information indicates 3 (3 in Sf_**1**), inter predictor **126** derives a motion vector using a mode in which an MV difference is encoded (for example, a normal inter mode (Sf_**5**).
(228) [MV Derivation>Normal Inter Model]
(229) The normal inter mode is an inter prediction mode for deriving an MV of a current block based on a block similar to the image of the current block from a reference picture region specified by an MV candidate. In this normal inter mode, an MV difference is encoded.
(230) FIG. **19** is a flow chart illustrating an example of inter prediction in normal inter mode.
(231) First, inter predictor **126** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially surrounding the current block (Step Sg_**1**). In other words, inter predictor **126** generates an MV candidate list.
(232) Next, inter predictor **126** extracts N (an integer of 2 or larger) MV candidates from the plurality of MV candidates obtained in Step Sg_**1**, as motion vector predictor candidates (also referred to as MV predictor candidates) according to a determined priority order (Step Sg_**2**). It is to be noted that the priority order may be determined in advance for each of the N MV candidates.
(233) Next, inter predictor **126** selects one motion vector predictor candidate from the N motion vector predictor candidates, as the motion vector predictor (also referred to as an MV predictor) of the current block (Step Sg_**3**). At this time, inter predictor **126** encodes, in a stream, motion vector predictor selection information for identifying the selected motion vector predictor. It is to be noted that the stream is an encoded signal or an encoded bitstream as described above.
(234) Next, inter predictor **126** derives an MV of a current block by referring to an encoded reference picture (Step Sg_**4**). At this time, inter predictor **126** further encodes, in the stream, the difference value between the derived MV and the motion vector predictor as an MV difference. It is to be noted that the encoded reference picture is a picture including a plurality of blocks which have been reconstructed after being encoded.
(235) Lastly, inter predictor **126** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Sg_**5**). It is to be noted that the prediction image is an inter prediction signal as described above.
(236) In addition, information indicating the inter prediction mode (normal inter mode in the above example) used to generate the prediction image is, for example, encoded as a prediction parameter.
(237) It is to be noted that the MV candidate list may be also used as a list for use in another mode. In addition, the processes related to the MV candidate list may be applied to processes related to the list for use in another mode. The processes related to the MV candidate list include, for example, extraction or selection of an MV candidate from the

MV candidate list, reordering of MV candidates, or deletion of an MV candidate.
(238) [MV Derivation>Merge Model]
(239) The merge mode is an inter prediction mode for selecting an MV candidate from an MV candidate list as an MV of a current block, thereby deriving the MV.
(240) FIG. **20** is a flow chart illustrating an example of inter prediction in merge mode.
(241) First, inter predictor **126** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially surrounding the current block (Step Sh_**1**). In other words, inter predictor **126** generates an MV candidate list.
(242) Next, inter predictor **126** selects one MV candidate from the plurality of MV candidates obtained in Step Sh_**1**, thereby deriving an MV of the current block (Step Sh_**2**). At this time, inter predictor **126** encodes, in a stream, MV selection information for identifying the selected MV candidate.
(243) Lastly, inter predictor **126** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Sh_**3**).
(244) In addition, information indicating the inter prediction mode (merge mode in the above example) used to generate the prediction image and included in the encoded signal is, for example, encoded as a prediction parameter.
(245) FIG. **21** is a conceptual diagram for illustrating one example of a motion vector derivation process of a current picture in merge mode.
(246) First, an MV candidate list in which MV predictor candidates are registered is generated. Examples of MV predictor candidates include: spatially neighboring MV predictors which are MVs of a plurality of encoded blocks located spatially surrounding a current block; temporally neighboring MV predictors which are MVs of surrounding blocks on which the position of a current block in an encoded reference picture is projected; combined MV predictors which are MVs generated by combining the MV value of a spatially neighboring MV predictor and the MV of a temporally neighboring MV predictor; and a zero MV predictor which is an MV having a zero value.
(247) Next, one MV predictor is selected from a plurality of MV predictors registered in an MV predictor list, and the selected MV predictor is determined as the MV of a current block.
(248) Furthermore, the variable length encoder describes and encodes, in a stream, merge_idx which is a signal indicating which MV predictor has been selected.
(249) It is to be noted that the MV predictors registered in the MV predictor list described in FIG. **21** are examples. The number of MV predictors may be different from the number of MV predictors in the diagram, and the MV predictor list may be configured in such a manner that some of the kinds of the MV predictors in the diagram may not be included, or that one or more MV predictors other than the kinds of MV predictors in the diagram are included.
(250) A final MV may be determined by performing a decoder motion vector refinement process (DMVR) to be described later using the MV of the current block derived in merge mode.
(251) It is to be noted that the MV predictor candidates are MV candidates described above, and the MV predictor list is the MV candidate list described above. It is to be noted that the MV candidate list may be referred to as a candidate list. In addition, merge_idx is MV selection information.
(252) [MV Derivation>FRUC Model]
(253) Motion information may be derived at the decoder side without being signaled from the encoder side. It is to be noted that, as described above, the merge mode defined in the H.265/HEVC standard may be used. In addition, for example, motion information may be derived by performing motion estimation at the decoder side. In an embodiment, at the decoder side, motion estimation is performed without using any pixel value in a current block.
(254) Here, a mode for performing motion estimation at the decoder side is described. The mode for performing motion estimation at the decoder side may be referred to as a pattern matched motion vector derivation (PMMVD) mode, or a frame rate up-conversion (FRUC) mode.
(255) One example of a FRUC process in the form of a flow chart is illustrated in FIG. **22**. First, a list of a plurality of candidates each having a motion vector (MV) predictor (that is, an MV candidate list that also may be used as a merge list) is generated by referring to a motion vector in an encoded block which spatially or temporally neighbors a current block (Step Si_**1**). Next, a best MV candidate is selected from the plurality of MV candidates registered in the MV candidate list (Step Si_**2**). For example, the evaluation values of the respective MV candidates included in the MV candidate list are calculated, and one MV candidate is selected based on the evaluation values. Based on the selected motion vector candidates, a motion vector for the current block is then derived (Step Si_**4**). More specifically, for example, the selected motion vector candidate (best MV candidate) is derived directly as the motion vector for the current block. In addition, for example, the motion vector for the current block may be derived using pattern matching in a surrounding region of a position in a reference picture where the position in the reference picture corresponds to the selected motion vector candidate. In other words, estimation using the pattern matching and the evaluation values may be performed in the surrounding region of the best MV candidate, and when there is an MV that yields a better evaluation value, the best MV candidate may be updated to the MV that yields the better evaluation value, and the updated MV may be determined as the final MV for the current block. A configuration in which no such a process for updating the best MV candidate to the MV having a better evaluation value is performed is also possible.
(256) Lastly, inter predictor **126** generates a prediction image for the current block by performing motion compensation

of the current block using the derived MV and the encoded reference picture (Step Si_**5**).
(257) A similar process may be performed in units of a sub-block.
(258) Evaluation values may be calculated according to various kinds of methods. For example, a comparison is made between a reconstructed image in a region in a reference picture corresponding to a motion vector, and a reconstructed image in a determined region (the region may be, for example, a region in another reference picture or a region in a neighboring block of a current picture, as indicated below). The determined region may be predetermined.
(259) The difference between the pixel values of the two reconstructed images may be used for an evaluation value of the motion vectors. It is to be noted that an evaluation value may be calculated using information other than the value of the difference.
(260) Next, an example of pattern matching is described in detail. First, one MV candidate included in an MV candidate list (for example, a merge list) is selected as a start point of estimation by the pattern matching. For example, as the pattern matching, either a first pattern matching or a second pattern matching may be used. The first pattern matching and the second pattern matching are also referred to as bilateral matching and template matching, respectively.
(261) [MV Derivation>FRUC>Bilateral Matching]
(262) In the first pattern matching, pattern matching is performed between two blocks along a motion trajectory of a current block which are two blocks in two different reference pictures. Accordingly, in the first pattern matching, a region in another reference picture along the motion trajectory of the current block is used as a determined region for calculating the evaluation value of the above-described candidate. The determined region may be predetermined.
(263) FIG. **23** is a conceptual diagram for illustrating one example of the first pattern matching (bilateral matching) between the two blocks in the two reference pictures along the motion trajectory. As illustrated in FIG. **23**, in the first pattern matching, two motion vectors (MV**0**, MV**1**) are derived by estimating a pair which best matches among pairs in the two blocks in the two different reference pictures (Ref**0**, Ref**1**) which are the two blocks along the motion trajectory of the current block (Cur block). More specifically, a difference between the reconstructed image at a specified location in the first encoded reference picture (Ref**0**) specified by an MV candidate, and the reconstructed image at a specified location in the second encoded reference picture (Ref**1**) specified by a symmetrical MV obtained by scaling the MV candidate at a display time interval is derived for the current block, and an evaluation value is calculated using the value of the obtained difference. It is possible to select, as the final MV, the MV candidate which yields the best evaluation value among the plurality of MV candidates, and which is likely to produce good results.
(264) In the assumption of a continuous motion trajectory, the motion vectors (MV**0**, MV**1**) specifying the two reference blocks are proportional to temporal distances (TD**0**, TD**1**) between the current picture (Cur Pic) and the two reference pictures (Ref**0**, Ref**1**). For example, when the current picture is temporally located between the two reference pictures and the temporal distances from the current picture to the respective two reference pictures are equal to each other, mirror-symmetrical bi-directional motion vectors are derived in the first pattern matching.
(265) [MV Derivation>FRUC>Template Matching]
(266) In the second pattern matching (template matching), pattern matching is performed between a block in a reference picture and a template in the current picture (the template is a block neighboring the current block in the current picture (the neighboring block is, for example, an upper and/or left neighboring block(s))). Accordingly, in the second pattern matching, the block neighboring the current block in the current picture is used as the determined region for calculating the evaluation value of the above-described candidate.
(267) FIG. **24** is a conceptual diagram for illustrating one example of pattern matching (template matching) between a template in a current picture and a block in a reference picture. As illustrated in FIG. **24**, in the second pattern matching, the motion vector of the current block (Cur block) is derived by estimating, in the reference picture (Ref**0**), the block which best matches the block neighboring the current block in the current picture (Cur Pic). More specifically, it is possible that the difference between a reconstructed image in an encoded region which neighbors both left and above or either left or above and a reconstructed image which is in a corresponding region in the encoded reference picture (Ref**0**) and is specified by an MV candidate is derived, an evaluation value is calculated using the value of the obtained difference, and the MV candidate which yields the best evaluation value among a plurality of MV candidates is selected as the best MV candidate.
(268) Such information indicating whether to apply the FRUC mode (referred to as, for example, a FRUC flag) may be signaled at the CU level. In addition, when the FRUC mode is applied (for example, when a FRUC flag is true), information indicating an applicable pattern matching method (either the first pattern matching or the second pattern matching) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, CTU level, or sub-block level).
(269) [MV Derivation>Affine Model]
(270) Next, the affine mode for deriving a motion vector in units of a sub-block based on motion vectors of a plurality of neighboring blocks is described. This mode is also referred to as an affine motion compensation prediction mode.
(271) FIG. **25**A is a conceptual diagram for illustrating one example of deriving a motion vector of each sub-block, based on motion vectors of a plurality of neighboring blocks. In FIG. **25**A, the current block includes sixteen 4×4 sub-blocks. Here, motion vector V**0** at an upper-left corner control point in the current block is derived based on a motion

vector of a neighboring block, and likewise, motion vector V**1** at an upper-right corner control point in the current block is derived based on a motion vector of a neighboring sub-block. Two motion vectors v**0** and v**1** may be projected according to an expression (1A) indicated below, and motion vectors (vx, vy) for the respective sub-blocks in the current block may be derived.

[Math. 1]

(272)
$$\begin{cases} v_x = \frac{(v_{1x} - v_{0x})}{w}x - \frac{(v_{1y} - v_{0y})}{w}y + v_{0x} \\ v_y = \frac{(v_{1y} - v_{0y})}{w}x + \frac{(v_{1x} - v_{0x})}{w}y + v_{0y} \end{cases} \quad (1A)$$

(273) Here, x and y indicate the horizontal position and the vertical position of the sub-block, respectively, and w indicates a determined weighting coefficient. The determined weighting coefficient may be predetermined.
(274) Such information indicating the affine mode (for example, referred to as an affine flag) may be signaled at the CU level. It is to be noted that the signaling of the information indicating the affine mode does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, CTU level, or sub-block level).
(275) In addition, the affine mode may include several modes for different methods for deriving motion vectors at the upper-left and upper-right corner control points. For example, the affine mode include two modes which are the affine inter mode (also referred to as an affine normal inter mode) and the affine merge mode.
(276) [MV Derivation>Affine Model]
(277) FIG. **25**B is a conceptual diagram for illustrating one example of deriving a motion vector of each sub-block in affine mode in which three control points are used. In FIG. **25**B, the current block includes sixteen 4×4 blocks. Here, motion vector V**0** at the upper-left corner control point for the current block is derived based on a motion vector of a neighboring block, and likewise, motion vector V**1** at the upper-right corner control point for the current block is derived based on a motion vector of a neighboring block, and motion vector V**2** at the lower-left corner control point for the current block is derived based on a motion vector of a neighboring block. Three motion vectors v**0**, v**1**, and v**2** may be projected according to an expression (1B) indicated below, and motion vectors (vx, vy) for the respective sub-blocks in the current block may be derived.

[Math. 2]

(278)
$$\begin{cases} v_x = \frac{(v_{1x} - v_{0x})}{w}x - \frac{(v_{2x} - v_{0x})}{h}y + v_{0x} \\ v_y = \frac{(v_{1y} - v_{0y})}{w}x - \frac{(v_{2y} - v_{0y})}{h}y + v_{0y} \end{cases} \quad (1B)$$

(279) Here, x and y indicate the horizontal position and the vertical position of the center of the sub-block, respectively, w indicates the width of the current block, and h indicates the height of the current block.
(280) Affine modes in which different numbers of control points (for example, two and three control points) are used may be switched and signaled at the CU level. It is to be noted that information indicating the number of control points in affine mode used at the CU level may be signaled at another level (for example, the sequence level, picture level, slice level, tile level, CTU level, or sub-block level).
(281) In addition, such an affine mode in which three control points are used may include different methods for deriving motion vectors at the upper-left, upper-right, and lower-left corner control points. For example, the affine modes include two modes which are the affine inter mode (also referred to as the affine normal inter mode) and the affine merge mode.
(282) [MV Derivation>Affine Merge Model]
(283) FIG. **26**A, FIG. **26**B, and FIG. **26**C are conceptual diagrams for illustrating the affine merge mode.
(284) As illustrated in FIG. **26**A, in the affine merge mode, for example, motion vector predictors at respective control points of a current block are calculated based on a plurality of motion vectors corresponding to blocks encoded according to the affine mode among encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) which neighbor the current block. More specifically, encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) are checked in the listed order, and the first effective block encoded according to the affine mode is identified. Motion vector predictors at the control points of the current block are calculated based on a plurality of motion vectors corresponding to the identified block.
(285) For example, as illustrated in FIG. **26**B, when block A which neighbors to the left of the current block has been encoded according to an affine mode in which two control points are used, motion vectors v**3** and v**4** projected at the upper-left corner position and the upper-right corner position of the encoded block including block A are derived. Motion vector predictor v**0** at the upper-left corner control point of the current block and motion vector predictor v**1** at the upper-right corner control point of the current block are then calculated from derived motion vectors v**3** and v**4**.
(286) For example, as illustrated in FIG. **26**C, when block A which neighbors to the left of the current block has been encoded according to an affine mode in which three control points are used, motion vectors v**3**, v**4**, and v**5** projected at the upper-left corner position, the upper-right corner position, and the lower-left corner position of the encoded block including block A are derived. Motion vector predictor v**0** at the upper-left corner control point of the current block, motion vector predictor v**1** at the upper-right corner control point of the current block, and motion vector predictor v**2** at the lower-left corner control point of the current block are then calculated from derived motion vectors v**3**, v**4**, and v**5**.

(287) It is to be noted that this method for deriving motion vector predictors may be used to derive motion vector predictors of the respective control points of the current block in Step Sj_**1** in FIG. **29** described later.
(288) FIG. **27** is a flow chart illustrating one example of the affine merge mode.
(289) In affine merge mode as illustrated, first, inter predictor **126** derives MV predictors of respective control points of a current block (Step Sk_**1**). The control points are an upper-left corner point of the current block and an upper-right corner point of the current block as illustrated in FIG. **25**A, or an upper-left corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. **25**B.
(290) In other words, as illustrated in FIG. **26**A, inter predictor **126** checks encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) in the listed order, and identifies the first effective block encoded according to the affine mode.
(291) When block A is identified and block A has two control points, as illustrated in FIG. **26**B, inter predictor **126** calculates motion vector v**0** at the upper-left corner control point of the current block and motion vector v**1** at the upper-right corner control point of the current block, from motion vectors v**3** and v**4** at the upper-left corner and the upper-right corner of the encoded block including block A. For example, inter predictor **126** calculates motion vector v**0** at the upper-left corner control point of the current block and motion vector v**1** at the upper-right corner control point of the current block, by projecting motion vectors v**3** and v**4** at the upper-left corner and the upper-right corner of the encoded block onto the current block.
(292) Alternatively, when block A is identified and block A has three control points, as illustrated in FIG. **26**C, inter predictor **126** calculates motion vector v**0** at the upper-left corner control point of the current block, motion vector v**1** at the upper-right corner control point of the current block, and motion vector v**2** at the lower-left corner control point of the current block from motion vectors v**3**, v**4**, and v**5** at the upper-left corner, the upper-right corner, and the lower-left corner of the encoded block including block A. For example, inter predictor **126** calculates motion vector v**0** at the upper-left corner control point of the current block, motion vector v**1** at the upper-right corner control point of the current block, and motion vector v**2** at the lower-left corner control point of the current block by projecting motion vectors v**3**, v**4**, and v**5** at the upper-left corner, the upper-right corner, and the lower-left corner of the encoded block onto the current block.
(293) Next, inter predictor **126** performs motion compensation of each of a plurality of sub-blocks included in the current block. In other words, inter predictor **126** calculates, for each of the plurality of sub-blocks, a motion vector of the sub-block as an affine MV, by using either (i) two motion vector predictors v**0** and v**1** and the expression (1A) described above or (ii) three motion vector predictors v**0**, v**1**, and v**2** and the expression (1B) described above (Step Sk_**2**). Inter predictor **126** then performs motion compensation of the sub-blocks using these affine MVs and encoded reference pictures (Step Sk_**3**). As a result, motion compensation of the current block is performed to generate a prediction image of the current block.
(294) [MV Derivation>Affine Inter Model]
(295) FIG. **28**A is a conceptual diagram for illustrating an affine inter mode in which two control points are used.
(296) In the affine inter mode, as illustrated in FIG. **28**A, a motion vector selected from motion vectors of encoded block A, block B, and block C which neighbor the current block is used as motion vector predictor v**0** at the upper-left corner control point of the current block. Likewise, a motion vector selected from motion vectors of encoded block D and block E which neighbor the current block is used as motion vector predictor v**1** at the upper-right corner control point of the current block.
(297) FIG. **28**B is a conceptual diagram for illustrating an affine inter mode in which three control points are used.
(298) In the affine inter mode, as illustrated in FIG. **28**B, a motion vector selected from motion vectors of encoded block A, block B, and block C which neighbor the current block is used as motion vector predictor v**0** at the upper-left corner control point of the current block. Likewise, a motion vector selected from motion vectors of encoded block D and block E which neighbor the current block is used as motion vector predictor v**1** at the upper-right corner control point of the current block. Furthermore, a motion vector selected from motion vectors of encoded block F and block G which neighbor the current block is used as motion vector predictor v**2** at the lower-left corner control point of the current block.
(299) FIG. **29** is a flow chart illustrating one example of an affine inter mode.
(300) In the affine inter mode as illustrated, first, inter predictor **126** derives MV predictors (v**0**, v**1**) or (v**0**, v**1**, v**2**) of respective two or three control points of a current block (Step Sj_**1**). The control points are an upper-left corner point of the current block and an upper-right corner point of the current block as illustrated in FIG. **25**A, or an upper-left corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. **25**B.
(301) In other words, inter predictor **126** derives the motion vector predictors (v**0**, v**1**) or (v**0**, v**1**, v**2**) of respective two or three control points of the current block by selecting motion vectors of any of the blocks among encoded blocks in the vicinity of the respective control points of the current block illustrated in either FIG. **28**A or FIG. **28**B. At this time, inter predictor **126** encodes, in a stream, motion vector predictor selection information for identifying the selected two motion vectors.
(302) For example, inter predictor **126** may determine, using a cost evaluation or the like, the block from which a

motion vector as a motion vector predictor at a control point is selected from among encoded blocks neighboring the current block, and may describe, in a bitstream, a flag indicating which motion vector predictor has been selected.

(303) Next, inter predictor **126** performs motion estimation (Step Sj_**3** and Sj_**4**) while updating a motion vector predictor selected or derived in Step Sj_**1** (Step Sj_**2**). In other words, inter predictor **126** calculates, as an affine MV, a motion vector of each of the sub-blocks which corresponds to an updated motion vector predictor, using either the expression (1A) or expression (1B) described above (Step Sj_**3**). Inter predictor **126** then performs motion compensation of the sub-blocks using these affine MVs and encoded reference pictures (Step Sj_**4**). As a result, for example, inter predictor **126** determines the motion vector predictor which yields the smallest cost as the motion vector at a control point in a motion estimation loop (Step Sj_**5**). At this time, inter predictor **126** further encodes, in the stream, the difference value between the determined MV and the motion vector predictor as an MV difference.

(304) Lastly, inter predictor **126** generates a prediction image for the current block by performing motion compensation of the current block using the determined MV and the encoded reference picture (Step Sj_**6**).

(305) [MV Derivation>Affine Inter Model]

(306) When affine modes in which different numbers of control points (for example, two and three control points) are used may be switched and signaled at the CU level, the number of control points in an encoded block and the number of control points in a current block may be different from each other. FIG. **30**A and FIG. **30**B are conceptual diagrams for illustrating methods for deriving motion vector predictors at control points when the number of control points in an encoded block and the number of control points in a current block are different from each other.

(307) For example, as illustrated in FIG. **30**A, when a current block has three control points at the upper-left corner, the upper-right corner, and the lower-left corner, and block A which neighbors to the left of the current block has been encoded according to an affine mode in which two control points are used, motion vectors v**3** and v**4** projected at the upper-left corner position and the upper-right corner position in the encoded block including block A are derived. Motion vector predictor v**0** at the upper-left corner control point of the current block and motion vector predictor v**1** at the upper-right corner control point of the current block are then calculated from derived motion vectors v**3** and v**4**. Furthermore, motion vector predictor v**2** at the lower-left corner control point is calculated from derived motion vectors v**0** and v**1**.

(308) For example, as illustrated in FIG. **30**B, when a current block has two control points at the upper-left corner and the upper-right corner, and block A which neighbors to the left of the current block has been encoded according to the affine mode in which three control points are used, motion vectors v**3**, v**4**, and v**5** projected at the upper-left corner position, the upper-right corner position, and the lower-left corner position in the encoded block including block A are derived. Motion vector predictor v**0** at the upper-left corner control point of the current block and motion vector predictor v**1** at the upper-right corner control point of the current block are then calculated from derived motion vectors v**3**, v**4**, and v**5**.

(309) It is to be noted that this method for deriving motion vector predictors may be used to derive motion vector predictors of the respective control points of the current block in Step Sj_**1** in FIG. **29**.

(310) [MV Derivation>DMVR]

(311) FIG. **31**A is a flow chart illustrating a relationship between the merge mode and DMVR.

(312) Inter predictor **126** derives a motion vector of a current block according to the merge mode (Step Sl_**1**). Next, inter predictor **126** determines whether to perform estimation of a motion vector, that is, motion estimation (Step Sl_**2**). Here, when determining not to perform motion estimation (No in Step Sl_**2**), inter predictor **126** determines the motion vector derived in Step Sl_**1** as the final motion vector for the current block (Step Sl_**4**). In other words, in this case, the motion vector of the current block is determined according to the merge mode.

(313) When determining to perform motion estimation in Step Sl_**1** (Yes in Step Sl_**2**), inter predictor **126** derives the final motion vector for the current block by estimating a surrounding region of the reference picture specified by the motion vector derived in Step Sl_**1** (Step Sl_**3**). In other words, in this case, the motion vector of the current block is determined according to the DMVR.

(314) FIG. **31**B is a conceptual diagram for illustrating one example of a DMVR process for determining an MV.

(315) First, (for example, in merge mode) the best MVP which has been set to the current block is determined to be an MV candidate. A reference pixel is identified from a first reference picture (L**0**) which is an encoded picture in the LO direction according to an MV candidate (LO). Likewise, a reference pixel is identified from a second reference picture (L**1**) which is an encoded picture in the L**1** direction according to an MV candidate (L**1**). A template is generated by calculating an average of these reference pixels.

(316) Next, each of the surrounding regions of MV candidates of the first reference picture (L**0**) and the second reference picture (L**1**) are estimated, and the MV which yields the smallest cost is determined to be the final MV. It is to be noted that the cost value may be calculated, for example, using a difference value between each of the pixel values in the template and a corresponding one of the pixel values in the estimation region, the values of MV candidates, etc.

(317) It is to be noted that the processes, configurations, and operations described here typically are basically common between the encoder and a decoder to be described later.

(318) Exactly the same example processes described here do not always need to be performed. Any process for enabling derivation of the final MV by estimation in surrounding regions of MV candidates may be used.

(319) [Motion Compensation>BIO/OBMC]
(320) Motion compensation involves a mode for generating a prediction image, and correcting the prediction image. The mode is, for example, BIO and OBMC to be described later.
(321) FIG. **32** is a flow chart illustrating one example of generation of a prediction image.
(322) Inter predictor **126** generates a prediction image (Step Sm_**1**), and corrects the prediction image, for example, according to any of the modes described above (Step Sm_**2**).
(323) FIG. **33** is a flow chart illustrating another example of generation of a prediction image.
(324) Inter predictor **126** determines a motion vector of a current block (Step Sn_**1**). Next, inter predictor **126** generates a prediction image (Step Sn_**2**), and determines whether to perform a correction process (Step Sn_**3**). Here, when determining to perform a correction process (Yes in Step Sn_**3**), inter predictor **126** generates the final prediction image by correcting the prediction image (Step Sn_**4**). When determining not to perform a correction process (No in Step Sn_**3**), inter predictor **126** outputs the prediction image as the final prediction image without correcting the prediction image (Step Sn_**5**).
(325) In addition, motion compensation involves a mode for correcting a luminance of a prediction image when generating the prediction image. The mode is, for example, LIC to be described later.
(326) FIG. **34** is a flow chart illustrating another example of generation of a prediction image.
(327) Inter predictor **126** derives a motion vector of a current block (Step So_**1**). Next, inter predictor **126** determines whether to perform a luminance correction process (Step So_**2**). Here, when determining to perform a luminance correction process (Yes in Step So_**2**), inter predictor **126** generates the prediction image while performing a luminance correction process (Step So_**3**). In other words, the prediction image is generated using LIC. When determining not to perform a luminance correction process (No in Step So_**2**), inter predictor **126** generates a prediction image by performing normal motion compensation without performing a luminance correction process (Step So_**4**).
(328) [Motion Compensation>OBMC]
(329) It is to be noted that an inter prediction signal may be generated using motion information for a neighboring block in addition to motion information for the current block obtained from motion estimation. More specifically, the inter prediction signal may be generated in units of a sub-block in the current block by performing a weighted addition of a prediction signal, based on motion information obtained from motion estimation (in the reference picture) and a prediction signal based on motion information for a neighboring block (in the current picture). Such inter prediction (motion compensation) is also referred to as overlapped block motion compensation (OBMC).
(330) In OBMC mode, information indicating a sub-block size for OBMC (referred to as, for example, an OBMC block size) may be signaled at the sequence level. Moreover, information indicating whether to apply the OBMC mode (referred to as, for example, an OBMC flag) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the sequence level and CU level, and may be performed at another level (for example, at the picture level, slice level, tile level, CTU level, or sub-block level).
(331) Examples of the OBMC mode will be described in further detail. FIGS. **35** and **36** are a flow chart and a conceptual diagram for illustrating an outline of a prediction image correction process performed by an OBMC process.
(332) First, as illustrated in FIG. **36**, a prediction image (Pred) is obtained through normal motion compensation using a motion vector (MV) assigned to the processing target (current) block. In FIG. **36**, the arrow “MV” points to a reference picture, and indicates what the current block of the current picture refers to in order to obtain a prediction image.
(333) Next, a prediction image (Pred_L) is obtained by applying a motion vector (MV_L) which has already been derived for the encoded block neighboring to the left of the current block to the current block (re-using the motion vector for the current block). The motion vector (MV_L) is indicated by an arrow “MV_L” indicating a reference picture from a current block. A first correction of a prediction image is performed by overlapping two prediction images Pred and Pred_L. This provides an effect of blending the boundary between neighboring blocks.
(334) Likewise, a prediction image (Pred_U) is obtained by applying a motion vector (MV_U) which has been already derived for the encoded block neighboring above the current block to the current block (re-using the motion vector for the current block). The motion vector (MV_U) is indicated by an arrow “MV_U” indicating a reference picture from a current block. A second correction of a prediction image is performed by overlapping the prediction image Pred_U to the prediction images (for example, Pred and Pred_L) on which the first correction has been performed. This provides an effect of blending the boundary between neighboring blocks. The prediction image obtained by the second correction is the one in which the boundary between the neighboring blocks has been blended (smoothed), and thus is the final prediction image of the current block.
(335) Although the above example is a two-path correction method using left and upper neighboring blocks, it is to be noted that the correction method may be three- or more-path correction method also using the right neighboring block and/or the lower neighboring block.
(336) It is to be noted that the region in which such overlapping is performed may be only part of a region near a block boundary instead of the pixel region of the entire block.
(337) It is to be noted that the prediction image correction process according to OBMC for obtaining one prediction image Pred from one reference picture by overlapping additional prediction image Pred_L and Pred_U have been described above. However, when a prediction image is corrected based on a plurality of reference images, a similar

process may be applied to each of the plurality of reference pictures. In such a case, after corrected prediction images are obtained from the respective reference pictures by performing OBMC image correction based on the plurality of reference pictures, the obtained corrected prediction images are further overlapped to obtain the final prediction image.
(338) It is to be noted that, in OBMC, the unit of a current block may be the unit of a prediction block or the unit of a sub-block obtained by further splitting the prediction block.
(339) One example of a method for determining whether to apply an OBMC process is a method for using an obmc_flag which is a signal indicating whether to apply an OBMC process. As one specific example, an encoder determines whether the current block belongs to a region having complicated motion. The encoder sets the obmc_flag to a value of "1" when the block belongs to a region having complicated motion and applies an OBMC process when encoding, and sets the obmc_flag to a value of "0" when the block does not belong to a region having complicated motion and encodes the block without applying an OBMC process. The decoder switches between application and non-application of an OBMC process by decoding the obmc_flag written in the stream (for example, a compressed sequence) and decoding the block by switching between the application and non-application of the OBMC process in accordance with the flag value.
(340) Inter predictor **126** generates one rectangular prediction image for a rectangular current block in the above example. However, inter predictor **126** may generate a plurality of prediction images each having a shape different from a rectangle for the rectangular current block, and may combine the plurality of prediction images to generate the final rectangular prediction image. The shape different from a rectangle may be, for example, a triangle.
(341) FIG. **37**A is a conceptual diagram for illustrating generation of two triangular prediction images.
(342) Inter predictor **126** generates a triangular prediction image by performing motion compensation of a first partition having a triangular shape in a current block by using a first MV of the first partition, to generate a triangular prediction image. Likewise, inter predictor **126** generates a triangular prediction image by performing motion compensation of a second partition having a triangular shape in a current block by using a second MV of the second partition, to generate a triangular prediction image. Inter predictor **126** then generates a prediction image having the same rectangular shape as the rectangular shape of the current block by combining these prediction images.
(343) It is to be noted that, although the first partition and the second partition are triangles in the example illustrated in FIG. **37**A, the first partition and the second partition may be trapezoids, or other shapes different from each other. Furthermore, although the current block includes two partitions in the example illustrated in FIG. **37**A, the current block may include three or more partitions.
(344) In addition, the first partition and the second partition may overlap with each other. In other words, the first partition and the second partition may include the same pixel region. In this case, a prediction image for a current block may be generated using a prediction image in the first partition and a prediction image in the second partition.
(345) FIG. **37**B is a conceptual diagram for illustrating examples of a first portion of a first partition which overlaps with a second partition, and first and second sets of samples which may be weighted as part of a correction process. The first portion may be, for example, one quarter of the width or height of the first partition. In another example, the first portion may have a width corresponding to N samples adjacent to an edge of the first partition, where N is an integer greater than zero, for example, N may be the integer 2. As illustrated, the left example of FIG. **37**B shows a rectangular partition having a rectangular portion with a width which is one fourth of the width of the first partition, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion. The center example of FIG. **37**B shows a rectangular partition having a rectangular portion with a height which is one fourth of the height of the first partition, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion. The right example of FIG. **37**B shows a triangular partition having a polygonal portion with a height which corresponds to two samples, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion.
(346) The first portion may be a portion of the first partition which overlaps with an adjacent partition. FIG. **37**C is a conceptual diagram for illustrating a first portion of a first partition, which is a portion of the first partition that overlaps with a portion of an adjacent partition. For ease of illustration, a rectangular partition having an overlapping portion with a spatially adjacent rectangular partition is shown. Partitions having other shapes, such as triangular partitions, may be employed, and the overlapping portions may overlap with a spatially or temporally adjacent partition.
(347) In addition, although an example is given in which a prediction image is generated for each of two partitions using inter prediction, a prediction image may be generated for at least one partition using intra prediction.
(348) [Motion Compensation>BIO]
(349) Next, a method for deriving a motion vector is described. First, a mode for deriving a motion vector based on a model assuming uniform linear motion will be described. This mode is also referred to as a bi-directional optical flow (BIO) mode.
(350) FIG. **38** is a conceptual diagram for illustrating a model assuming uniform linear motion. In FIG. **38**, (vx, vy) indicates a velocity vector, and τ**0** and τ**1** indicate temporal distances between a current picture (Cur Pic) and two reference pictures (Ref**0**, Ref**1**). (MVx**0**, MVy**0**) indicate motion vectors corresponding to reference picture Ref**0**, and

(MVx**1**, MVy**1**) indicate motion vectors corresponding to reference picture Ref**1**.
(351) Here, under the assumption of uniform linear motion exhibited by velocity vectors (vx, vy), (MVx**0**, MVy**0**) and (MVx**1**, MVy**1**) are represented as (vxτ**0**, vyτ**0**) and (−vxτ**1**, −vyτ**1**), respectively, and the following optical flow equation (2) may be employed.

[MATH. 3]

(352) $$\partial I^{(k)} / \partial t + v_x \, \partial I^{(k)} / \partial x + v_y \, \partial I^{(k)} / \partial y = 0. \quad (2)$$

(353) Here, I(k) indicates a motion-compensated luma value of reference picture k (k=0, 1). This optical flow equation shows that the sum of (i) the time derivative of the luma value, (ii) the product of the horizontal velocity and the horizontal component of the spatial gradient of a reference image, and (iii) the product of the vertical velocity and the vertical component of the spatial gradient of a reference image is equal to zero. A motion vector of each block obtained from, for example, a merge list may be corrected in units of a pixel, based on a combination of the optical flow equation and Hermite interpolation.
(354) It is to be noted that a motion vector may be derived on the decoder side using a method other than deriving a motion vector based on a model assuming uniform linear motion. For example, a motion vector may be derived in units of a sub-block based on motion vectors of neighboring blocks.
(355) [Motion Compensation>LIC]
(356) Next, an example of a mode in which a prediction image (prediction) is generated by using a local illumination compensation (LIC) process will be described.
(357) FIG. **39** is a conceptual diagram for illustrating one example of a prediction image generation method using a luminance correction process performed by a LIC process.
(358) First, an MV is derived from an encoded reference picture, and a reference image corresponding to the current block is obtained.
(359) Next, information indicating how the luma value changed between the reference picture and the current picture is extracted for the current block. This extraction is performed based on the luma pixel values for the encoded left neighboring reference region (surrounding reference region) and the encoded upper neighboring reference region (surrounding reference region), and the luma pixel value at the corresponding position in the reference picture specified by the derived MV. A luminance correction parameter is calculated by using the information indicating how the luma value changed.
(360) The prediction image for the current block is generated by performing a luminance correction process in which the luminance correction parameter is applied to the reference image in the reference picture specified by the MV.
(361) It is to be noted that the shape of the surrounding reference region illustrated in FIG. **39** is just one example; the surrounding reference region may have a different shape.
(362) Moreover, although the process in which a prediction image is generated from a single reference picture has been described here, cases in which a prediction image is generated from a plurality of reference pictures can be described in the same manner. The prediction image may be generated after performing a luminance correction process of the reference images obtained from the reference pictures in the same manner as described above.
(363) One example of a method for determining whether to apply a LIC process is a method for using a lic_flag which is a signal indicating whether to apply the LIC process. As one specific example, the encoder determines whether the current block belongs to a region having a luminance change. The encoder sets the lic_flag to a value of “1” when the block belongs to a region having a luminance change and applies a LIC process when encoding, and sets the lic_flag to a value of “0” when the block does not belong to a region having a luminance change and encodes the current block without applying a LIC process. The decoder may decode the lic_flag written in the stream and decode the current block by switching between application and non-application of a LIC process in accordance with the flag value.
(364) One example of a different method of determining whether to apply a LIC process is a determining method in accordance with whether a LIC process was applied to a surrounding block. In one specific example, when the merge mode is used on the current block, whether a LIC process was applied in the encoding of the surrounding encoded block selected upon deriving the MV in the merge mode process is determined. According to the result, encoding is performed by switching between application and non-application of a LIC process. It is to be noted that, also in this example, the same processes are applied in processes at the decoder side.
(365) An embodiment of the luminance correction (LIC) process described with reference to FIG. **39** is described in detail below.
(366) First, inter predictor **126** derives a motion vector for obtaining a reference image corresponding to a current block to be encoded from a reference picture which is an encoded picture.
(367) Next, inter predictor **126** extracts information indicating how the luma value of the reference picture has been changed to the luma value of the current picture, using the luma pixel value of an encoded surrounding reference region which neighbors to the left of or above the current block, and the luma value in the corresponding position in the reference picture specified by a motion vector, and calculates a luminance correction parameter. For example, it is assumed that the luma pixel value of a given pixel in the surrounding reference region in the current picture is p**0**, and that the luma pixel value of the pixel corresponding to the given pixel in the surrounding reference region in the

reference picture is p**1**. Inter predictor **126** calculates coefficients A and B for optimizing A×p**1**+B=p**0** as the luminance correction parameter for a plurality of pixels in the surrounding reference region.
(368) Next, inter predictor **126** performs a luminance correction process using the luminance correction parameter for the reference image in the reference picture specified by the motion vector, to generate a prediction image for the current block. For example, it is assumed that the luma pixel value in the reference image is p**2**, and that the luminance-corrected luma pixel value of the prediction image is p**3**. Inter predictor **126** generates the prediction image after being subjected to the luminance correction process by calculating A×p**2**+B=p**3** for each of the pixels in the reference image.
(369) It is to be noted that the shape of the surrounding reference region illustrated in FIG. **39** is one example; a different shape other than the shape of the surrounding reference region may be used. In addition, part of the surrounding reference region illustrated in FIG. **39** may be used. For example, a region having a determined number of pixels extracted from each of an upper neighboring pixel and a left neighboring pixel may be used as a surrounding reference region. The determined number of pixels may be predetermined.
(370) In addition, the surrounding reference region is not limited to a region which neighbors the current block, and may be a region which does not neighbor the current block. In the example illustrated in FIG. **39**, the surrounding reference region in the reference picture is a region specified by a motion vector in a current picture, from a surrounding reference region in the current picture. However, a region specified by another motion vector is also possible. For example, the other motion vector may be a motion vector in a surrounding reference region in the current picture.
(371) Although operations performed by encoder **100** have been described here, it is to be noted that decoder **200** typically performs similar operations.
(372) It is to be noted that the LIC process may be applied not only to the luma but also to chroma. At this time, a correction parameter may be derived individually for each of Y, Cb, and Cr, or a common correction parameter may be used for any of Y, Cb, and Cr.
(373) In addition, the LIC process may be applied in units of a sub-block. For example, a correction parameter may be derived using a surrounding reference region in a current sub-block and a surrounding reference region in a reference sub-block in a reference picture specified by an MV of the current sub-block.
(374) [Prediction Controller]
(375) Inter predictor **128** selects one of an intra prediction signal (a signal output from intra predictor **124**) and an inter prediction signal (a signal output from inter predictor **126**), and outputs the selected signal to subtractor **104** and adder **116** as a prediction signal.
(376) As illustrated in FIG. **1**, in various kinds of encoder examples, prediction controller **128** may output a prediction parameter which is input to entropy encoder **110**. Entropy encoder **110** may generate an encoded bitstream (or a sequence), based on the prediction parameter which is input from prediction controller **128** and quantized coefficients which are input from quantizer **108**. The prediction parameter may be used in a decoder. The decoder may receive and decode the encoded bitstream, and perform the same processes as the prediction processes performed by intra predictor **124**, inter predictor **126**, and prediction controller **128**. The prediction parameter may include (i) a selection prediction signal (for example, a motion vector, a prediction type, or a prediction mode used by intra predictor **124** or inter predictor **126**), or (ii) an optional index, a flag, or a value which is based on a prediction process performed in each of intra predictor **124**, inter predictor **126**, and prediction controller **128**, or which indicates the prediction process.
(377) [Mounting Example of Encoder]
(378) FIG. **40** is a block diagram illustrating a mounting example of an encoder **100**. Encoder **100** includes processor a**1** and memory a**2**. For example, the plurality of constituent elements of encoder **100** illustrated in FIG. **1** are mounted on processor a**1** and memory a**2** illustrated in FIG. **40**.
(379) Processor a**1** is circuitry which performs information processing and is accessible to memory a**2**. For example, processor a**1** is dedicated or general electronic circuitry which encodes a video. Processor a**1** may be a processor such as a CPU. In addition, processor a**1** may be an aggregate of a plurality of electronic circuits. In addition, for example, processor a**1** may take the roles of two or more constituent elements out of the plurality of constituent elements of encoder **100** illustrated in FIG. **1**, etc.
(380) Memory a**2** is dedicated or general memory for storing information that is used by processor a**1** to encode a video. Memory a**2** may be electronic circuitry, and may be connected to processor a**1**. In addition, memory a**2** may be included in processor a**1**. In addition, memory a**2** may be an aggregate of a plurality of electronic circuits. In addition, memory a**2** may be a magnetic disc, an optical disc, or the like, or may be represented as a storage, a recording medium, or the like. In addition, memory a**2** may be non-volatile memory, or volatile memory.
(381) For example, memory a**2** may store a video to be encoded or a bitstream corresponding to an encoded video. In addition, memory a**2** may store a program for causing processor a**1** to encode a video.
(382) In addition, for example, memory a**2** may take the roles of two or more constituent elements for storing information out of the plurality of constituent elements of encoder **100** illustrated in FIG. **1**, etc. For example, memory a**2** may take the roles of block memory **118** and frame memory **122** illustrated in FIG. **1**. More specifically, memory a**2** may store a reconstructed block, a reconstructed picture, etc.
(383) It is to be noted that, in encoder **100**, all of the plurality of constituent elements indicated in FIG. **1**, etc. may not be implemented, and all the processes described above may not be performed. Part of the constituent elements indicated

in FIG. **1**, etc. may be included in another device, or part of the processes described above may be performed by another device.
(384) [Decoder]
(385) Next, a decoder capable of decoding an encoded signal (encoded bitstream) output, for example, from encoder **100** described above will be described. FIG. **41** is a block diagram illustrating a functional configuration of decoder **200** according to an embodiment. Decoder **200** is a video decoder which decodes a video in units of a block.
(386) As illustrated in FIG. **41**, decoder **200** includes entropy decoder **202**, inverse quantizer **204**, inverse transformer **206**, adder **208**, block memory **210**, loop filter **212**, frame memory **214**, intra predictor **216**, inter predictor **218**, and prediction controller **220**.
(387) Decoder **200** is implemented as, for example, a generic processor and memory. In this case, when a software program stored in the memory is executed by the processor, the processor functions as entropy decoder **202**, inverse quantizer **204**, inverse transformer **206**, adder **208**, loop filter **212**, intra predictor **216**, inter predictor **218**, and prediction controller **220**. Alternatively, decoder **200** may be implemented as one or more dedicated electronic circuits corresponding to entropy decoder **202**, inverse quantizer **204**, inverse transformer **206**, adder **208**, loop filter **212**, intra predictor **216**, inter predictor **218**, and prediction controller **220**.
(388) Hereinafter, an overall flow of processes performed by decoder **200** is described, and then each of constituent elements included in decoder **200** will be described.
(389) [Overall Flow of Decoding Process]
(390) FIG. **42** is a flow chart illustrating one example of an overall decoding process performed by decoder **200**.
(391) First, entropy decoder **202** of decoder **200** identifies a splitting pattern of a block having a fixed size (for example, 128×128 pixels) (Step Sp_**1**). This splitting pattern is a splitting pattern selected by encoder **100**. Decoder **200** then performs processes of Step Sp_**2** to Sp_**6** for each of a plurality of blocks of the splitting pattern.
(392) In other words, entropy decoder **202** decodes (specifically, entropy-decodes) encoded quantized coefficients and a prediction parameter of a current block to be decoded (also referred to as a current block) (Step Sp_**2**).
(393) Next, inverse quantizer **204** performs inverse quantization of the plurality of quantized coefficients, and inverse transformer **206** performs inverse transform of the result, to restore a plurality of prediction residuals (that is, a difference block) (Step Sp_**3**).
(394) Next, the prediction processor including all or part of intra predictor **216**, inter predictor **218**, and prediction controller **220** generates a prediction signal (also referred to as a prediction block) of the current block (Step Sp_**4**).
(395) Next, adder **208** adds the prediction block to the difference block to generate a reconstructed image (also referred to as a decoded image block) of the current block (Step Sp_**5**).
(396) When the reconstructed image is generated, loop filter **212** performs filtering of the reconstructed image (Step Sp_**6**).
(397) Decoder **200** then determines whether decoding of the entire picture has been finished (Step Sp_**7**). When determining that the decoding has not yet been finished (No in Step Sp_**7**), decoder **200** repeatedly executes the processes starting with Step Sp_**1**.
(398) As illustrated, the processes of Steps Sp_**1** to Sp_**7** are performed sequentially by decoder **200**, Alternatively, two or more of the processes may be performed in parallel, the processing order of the two or more of the processes may be modified, etc.
(399) [Entropy Decoder]
(400) Entropy decoder **202** entropy decodes an encoded bitstream. More specifically, for example, entropy decoder **202** arithmetic decodes an encoded bitstream into a binary signal. Entropy decoder **202** then debinarizes the binary signal. With this, entropy decoder **202** outputs quantized coefficients of each block to inverse quantizer **204**. Entropy decoder **202** may output a prediction parameter included in an encoded bitstream (see FIG. **1**) to intra predictor **216**, inter predictor **218**, and prediction controller **220**. Intra predictor **216**, inter predictor **218**, and prediction controller **220** in an embodiment are capable of executing the same prediction processes as those performed by intra predictor **124**, inter predictor **126**, and prediction controller **128** at the encoder side.
(401) [Inverse Quantizer]
(402) Inverse quantizer **204** inverse quantizes quantized coefficients of a block to be decoded (hereinafter referred to as a current block) which are inputs from entropy decoder **202**. More specifically, inverse quantizer **204** inverse quantizes quantized coefficients of the current block, based on quantization parameters corresponding to the quantized coefficients. Inverse quantizer **204** then outputs the inverse quantized transform coefficients of the current block to inverse transformer **206**.
(403) [Inverse Transformer]
(404) Inverse transformer **206** restores prediction errors by inverse transforming the transform coefficients which are inputs from inverse quantizer **204**.
(405) For example, when information parsed from an encoded bitstream indicates that EMT or AMT is to be applied (for example, when an AMT flag is true), inverse transformer **206** inverse transforms the transform coefficients of the current block based on information indicating the parsed transform type.
(406) Moreover, for example, when information parsed from an encoded bitstream indicates that NSST is to be applied,

inverse transformer **206** applies a secondary inverse transform to the transform coefficients.
(407) [Adder]
(408) Adder **208** reconstructs the current block by adding prediction errors which are inputs from inverse transformer **206** and prediction samples which are inputs from prediction controller **220**. Adder **208** then outputs the reconstructed block to block memory **210** and loop filter **212**.
(409) [Block Memory]
(410) Block memory **210** is storage for storing blocks in a picture to be decoded (hereinafter referred to as a current picture) and to be referred to in intra prediction. More specifically, block memory **210** stores reconstructed blocks output from adder **208**.
(411) [Loop Filter]
(412) Loop filter **212** applies a loop filter to blocks reconstructed by adder **208**, and outputs the filtered reconstructed blocks to frame memory **214**, display device, etc.
(413) When information indicating ON or OFF of an ALF parsed from an encoded bitstream indicates that an ALF is ON, one filter from among a plurality of filters is selected based on direction and activity of local gradients, and the selected filter is applied to the reconstructed block.
(414) [Frame Memory]
(415) Frame memory **214** is, for example, storage for storing reference pictures for use in inter prediction, and is also referred to as a frame buffer. More specifically, frame memory **214** stores a reconstructed block filtered by loop filter **212**.
(416) [Prediction Processor (Intra Predictor, Inter Predictor, Prediction Controller)]
(417) FIG. **43** is a flow chart illustrating one example of a process performed by a prediction processor of decoder **200**. It is to be noted that the prediction processor includes all or part of the following constituent elements: intra predictor **216**; inter predictor **218**; and prediction controller **220**.
(418) The prediction processor generates a prediction image of a current block (Step Sq_**1**). This prediction image is also referred to as a prediction signal or a prediction block. It is to be noted that the prediction signal is, for example, an intra prediction signal or an inter prediction signal. Specifically, the prediction processor generates the prediction image of the current block using a reconstructed image which has been already obtained through generation of a prediction block, generation of a difference block, generation of a coefficient block, restoring of a difference block, and generation of a decoded image block.
(419) The reconstructed image may be, for example, an image in a reference picture, or an image of a decoded block in a current picture which is the picture including the current block. The decoded block in the current picture is, for example, a neighboring block of the current block.
(420) FIG. **44** is a flow chart illustrating another example of a process performed by the prediction processor of decoder **200**.
(421) The prediction processor determines either a method or a mode for generating a prediction image (Step Sr_**1**). For example, the method or mode may be determined based on, for example, a prediction parameter, etc.
(422) When determining a first method as a mode for generating a prediction image, the prediction processor generates a prediction image according to the first method (Step Sr_**2***a*). When determining a second method as a mode for generating a prediction image, the prediction processor generates a prediction image according to the second method (Step Sr_**2***b*). When determining a third method as a mode for generating a prediction image, the prediction processor generates a prediction image according to the third method (Step Sr_**2***c*).
(423) The first method, the second method, and the third method may be mutually different methods for generating a prediction image. Each of the first to third methods may be an inter prediction method, an intra prediction method, or another prediction method. The above-described reconstructed image may be used in these prediction methods.
(424) [Intra Predictor]
(425) Intra predictor **216** generates a prediction signal (intra prediction signal) by performing intra prediction by referring to a block or blocks in the current picture stored in block memory **210**, based on the intra prediction mode parsed from the encoded bitstream. More specifically, intra predictor **216** generates an intra prediction signal by performing intra prediction by referring to samples (for example, luma and/or chroma values) of a block or blocks neighboring the current block, and then outputs the intra prediction signal to prediction controller **220**.
(426) It is to be noted that when an intra prediction mode in which a luma block is referred to in intra prediction of a chroma block is selected, intra predictor **216** may predict the chroma component of the current block based on the luma component of the current block.
(427) Moreover, when information parsed from an encoded bitstream indicates that PDPC is to be applied, intra predictor **216** corrects intra-predicted pixel values based on horizontal/vertical reference pixel gradients.
(428) [Inter Predictor]
(429) Inter predictor **218** predicts the current block by referring to a reference picture stored in frame memory **214**. Inter prediction is performed in units of a current block or a sub-block (for example, a 4×4 block) in the current block. For example, inter predictor **218** generates an inter prediction signal of the current block or the sub-block by performing motion compensation by using motion information (for example, a motion vector) parsed from an encoded bitstream

(for example, a prediction parameter output from entropy decoder **202**), and outputs the inter prediction signal to prediction controller **220**.
(430) It is to be noted that when the information parsed from the encoded bitstream indicates that the OBMC mode is to be applied, inter predictor **218** generates the inter prediction signal using motion information of a neighboring block in addition to motion information of the current block obtained from motion estimation.
(431) Moreover, when the information parsed from the encoded bitstream indicates that the FRUC mode is to be applied, inter predictor **218** derives motion information by performing motion estimation in accordance with the pattern matching method (bilateral matching or template matching) parsed from the encoded bitstream. Inter predictor **218** then performs motion compensation (prediction) using the derived motion information.
(432) Moreover, when the BIO mode is to be applied, inter predictor **218** derives a motion vector based on a model assuming uniform linear motion. Moreover, when the information parsed from the encoded bitstream indicates that the affine motion compensation prediction mode is to be applied, inter predictor **218** derives a motion vector of each sub-block based on motion vectors of neighboring blocks.
(433) [MV Derivation>Normal Inter Model]
(434) When information parsed from an encoded bitstream indicates that the normal inter mode is to be applied, inter predictor **218** derives an MV based on the information parsed from the encoded bitstream and performs motion compensation (prediction) using the MV.
(435) FIG. **45** is a flow chart illustrating an example of inter prediction in normal inter mode in decoder **200**.
(436) Inter predictor **218** of decoder **200** performs motion compensation for each block. Inter predictor **218** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of decoded blocks temporally or spatially surrounding the current block (Step Ss_**1**). In other words, inter predictor **218** generates an MV candidate list.
(437) Next, inter predictor **218** extracts N (an integer of 2 or larger) MV candidates from the plurality of MV candidates obtained in Step Ss_**1**, as motion vector predictor candidates (also referred to as MV predictor candidates) according to a determined priority order (Step Ss_**2**). It is to be noted that the priority order may be determined in advance for each of the N MV predictor candidates.
(438) Next, inter predictor **218** decodes motion vector predictor selection information from an input stream (that is, an encoded bitstream), and selects one MV predictor candidate, from the N MV predictor candidates using the decoded motion vector predictor selection information, as a motion vector (also referred to as an MV predictor) of the current block (Step Ss_**3**).
(439) Next, inter predictor **218** decodes an MV difference from the input stream, and derives an MV for a current block by adding a difference value which is the decoded MV difference and a selected motion vector predictor (Step Ss_**4**).
(440) Lastly, inter predictor **218** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the decoded reference picture (Step Ss_**5**).
(441) [Prediction Controller]
(442) Prediction controller **220** selects either the intra prediction signal or the inter prediction signal, and outputs the selected prediction signal to adder **208**. As a whole, the configurations, functions, and processes of prediction controller **220**, intra predictor **216**, and inter predictor **218** at the decoder side may correspond to the configurations, functions, and processes of prediction controller **128**, intra predictor **124**, and inter predictor **126** at the encoder side. It is to be noted that all or part of the following constituent elements may be included in a predictor: intra predictor **216**; inter predictor **218**; and prediction controller **220**.
(443) [Mounting Example of Decoder]
(444) FIG. **46** is a block diagram illustrating a mounting example of a decoder **200**. Decoder **200** includes processor b**1** and memory b**2**. For example, the plurality of constituent elements of decoder **200** illustrated in FIG. **41** are mounted on processor b**1** and memory b**2** illustrated in FIG. **46**.
(445) Processor b**1** is circuitry which performs information processing and is accessible to memory b**2**. For example, processor b**1** is dedicated or general electronic circuitry which decodes a video (that is, an encoded bitstream). Processor b**1** may be a processor such as a CPU. In addition, processor b**1** may be an aggregate of a plurality of electronic circuits. In addition, for example, processor b**1** may take the roles of two or more constituent elements out of the plurality of constituent elements of decoder **200** illustrated in FIG. **41**, etc.
(446) Memory b**2** is dedicated or general memory for storing information that is used by processor b**1** to decode an encoded bitstream. Memory b**2** may be electronic circuitry, and may be connected to processor b**1**. In addition, memory b**2** may be included in processor b**1**. In addition, memory b**2** may be an aggregate of a plurality of electronic circuits. In addition, memory b**2** may be a magnetic disc, an optical disc, or the like, or may be represented as storage, a recording medium, or the like. In addition, memory b**2** may be a non-volatile memory, or a volatile memory.
(447) For example, memory b**2** may store a video or a bitstream. In addition, memory b**2** may store a program for causing processor b**1** to decode an encoded bitstream.
(448) In addition, for example, memory b**2** may take the roles of two or more constituent elements for storing information out of the plurality of constituent elements of decoder **200** illustrated in FIG. **41**, etc. Specifically, memory b**2** may take the roles of block memory **210** and frame memory **214** illustrated in FIG. **41**. More specifically, memory

b**2** may store a reconstructed block, a reconstructed picture, etc.
(449) It is to be noted that, in decoder **200**, all of the plurality of constituent elements illustrated in FIG. **41**, etc. may not be implemented, and all the processes described above may not be performed. Part of the constituent elements indicated in FIG. **41**, etc. may be included in another device, or part of the processes described above may be performed by another device.
Definitions of Terms
(450) The respective terms may be defined as indicated below as examples.
(451) A picture is an array of luma samples in monochrome format or an array of luma samples and two corresponding arrays of chroma samples in 4:2:0, 4:2:2, and 4:4:4 color format. A picture may be either a frame or a field.
(452) A frame is the composition of a top field and a bottom field, where sample rows 0, 2, 4, . . . originate from the top field and sample rows 1, 3, 5, . . . originate from the bottom field.
(453) A slice is an integer number of coding tree units contained in one independent slice segment and all subsequent dependent slice segments (if any) that precede the next independent slice segment (if any) within the same access unit.
(454) A tile is a rectangular region of coding tree blocks within a particular tile column and a particular tile row in a picture. A tile may be a rectangular region of the frame that is intended to be able to be decoded and encoded independently, although loop-filtering across tile edges may still be applied.
(455) A block is an M×N (M-column by N-row) array of samples, or an M×N array of transform coefficients. A block may be a square or rectangular region of pixels including one Luma and two Chroma matrices.
(456) A coding tree unit (CTU) may be a coding tree block of luma samples of a picture that has three sample arrays, or two corresponding coding tree blocks of chroma samples. Alternatively, a CTU may be a coding tree block of samples of one of a monochrome picture and a picture that is coded using three separate color planes and syntax structures used to code the samples. A super block may be a square block of 64×64 pixels that consists of either 1 or 2 mode info blocks or is recursively partitioned into four 32×32 blocks, which themselves can be further partitioned.
(457) [One Example of Internal Structure of Predictor]
(458) As one example, inter predictor **126** and **218** may perform a prediction process referencing decoded pixels in a current picture including a current block in addition to pictures located backward or forward in a time direction. This prediction process may be called Intra Block Copy (IBC) and references pixels inside the current picture. IBC may be implemented as an inter prediction mode similar to other inter modes, for example, merge mode, normal inter mode, and skip mode.
(459) FIG. **47**A illustrates one example of locations of a current block and a reference area in a current picture. The reference area includes reference pixels and may be located on the upper side and the left side of the current block so that already decoded pixels are referenced by the current block. As illustrated, the current block has a height Hc and a width Wc.
(460) FIG. **47**B illustrates one example of a relationship between a reference block and the current block. The current block has a width Wc and a height Hc. A vector indicating displacement inside a picture between the reference block and the current block may be called a Block Vector (BV). A starting point of the block vector BV is, as one example, a pixel location of a top right corner of the current block. In this case, BV is a vector connecting the top right corner of the current block and a top right corner of the reference block. As another example, the starting point of the block vector BV may be a pixel location of a top left corner of the current block. In this case, the block vector BV is a vector connecting the top left corner of the current block and a top left corner of the reference block. The block vector BV may be expressed using X component BVx and Y component BVy of the block vector BV.
(461) [First Aspect]
(462) FIG. **48** is a flow chart illustrating one example of a process of decoding an image block using a prediction mode selected from a plurality of prediction modes including inter mode, intra mode, and IBC mode. The prediction process as illustrated in FIG. **48** may be performed by an encoder and a decoder.
(463) In step S**1001**, a decoder determines a size of an image block to be processed (hereinafter also referred to as a current block). As one example, the size of the current block may be at least one of width and height of the current block. As another example, the size of the current block may be both of width and height of the current block. The decoder also may determine whether IBC mode may be employed to perform prediction for the current block. For example, the decoder may determine whether a flag indicates IBC mode is allowed to be applied to a sequence including the current block. As another example, the decoder may determine the size of the block in response to IBC mode being potentially applicable to the current block.
(464) In step S**1002** and step S**1003**, a decoder determines whether the size of the current block satisfies a size condition. For example, the size condition may be a first condition that at least one of width and height of the current block is greater than a first value. The first value is a threshold value and may be 64 as an example. The size condition may be a second condition that both of the width and the height of the current block are less than or equal to the first value.
(465) In step S**1004**, in response to the size of the current block not satisfying the first condition (in other words, in response to the size of the current block satisfying the second condition), IBC mode may be applied to the current block to generate the prediction image. In other words, a prediction mode may be selected from a plurality of prediction

modes including IBC mode, another inter mode, and an intra mode when both of width and height of the current block are less than or equal to 64. In step S**1004**, a flag indicating whether IBC mode is to be used for the current block may be decoded from a bitstream.
(466) In step S**1005**, IBC mode is disabled in response to the size of the current block satisfying the first condition (in other words, in response to the size of the current block not satisfying the second condition). In other words, IBC mode is not included in the plurality of prediction modes that may be selected when at least one of width and height is greater than 64. The current block may be decoded using another inter mode or an intra mode. It is not necessary to decode a flag indicating whether IBC mode is used for the current block.
(467) As discussed above, IBC mode is a mode for deriving a predicted image by referring to the decoded area of the current picture.
(468) One of skill in the art will recognize that other size conditions may be employed. For example, the first threshold value may be different value, such as 32.
(469) As described above, according to one embodiment, an image encoder is provided including circuitry and a memory coupled to the circuitry. The circuitry, in operation, performs: in response to a size of a block satisfying a size condition (FIG. **48**, step S**1002** and S**1003**), generating a prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode (step S**1004**), the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block; and encoding the block using the prediction image.
(470) The present disclosure may introduce a size constraint for IBC mode, which may reduce on-chip buffer needs. This may facilitate more efficient use of resources (such as buffer area) and more efficient processing of images.
(471) According to an embodiment, the first prediction mode is intra block copy mode (see FIGS. **47**A and **47**B and the description thereof). According to an embodiment, the size condition is that both of width and height of the block is less than or equal to a first threshold value. According to an embodiment, the first threshold value is 64 (step S**1002** and S**1003**). According to an embodiment, the circuitry codes (or decodes) a flag indicating whether the first prediction mode (step S**1004**) is selected. According to an embodiment, the circuitry, in response to the size of the block not satisfying the size condition, generates the prediction image using a prediction mode selected from a plurality of prediction modes excluding the first prediction mode (step S**1005**).
(472) All processes/elements described above are not always needed. The device/method may contain a portion of the processes/elements. The process described above may be performed by a decoder or an encoder.
(473) [Second Aspect]
(474) FIG. **49** is a flow chart illustrating one example of a process of decoding an image block using a prediction mode selected from a plurality of prediction modes including inter mode, intra mode, and IBC mode. The prediction process as illustrated in FIG. **49** may be performed by an encoder and a decoder.
(475) In step S**2001**, a decoder determines a size of a current block. As one example, the size of the current block may be at least one of width and height of the current block. As another example, the size of the current block may be both of width and height of the current block. The decoder also may determine whether IBC mode may be employed to predict images for the current block. For example, the decoder may determine whether a flag indicates IBC mode is allowed to be applied to a sequence including the current block. As another example, the decoder may determine the size of the block in response to IBC mode being potentially applicable to the current block.
(476) In step S**2002** and step S**2003**, a decoder determines whether the size of the current block satisfies a size condition. For example, the size condition may be a third condition that both of width and height of the current block are equal to a second value. The second value is a threshold value and may be 128 as an example. The size condition may be a fourth condition that at least one of width and height of the current block is not equal to the second value.
(477) In step S**2004**, in response to the size of the current block not satisfying the third condition (in other words, in response to the size of the current block satisfying the fourth condition), IBC mode may be applied to the current block to generate the prediction image. In other words, a prediction mode may be selected from a plurality of prediction modes including IBC mode, another inter mode, and an intra mode when at least one of width and height of the current block is not equal to 128. In step S**2004**, a flag indicating whether IBC mode is to be used for the current block may be decoded from a bitstream.
(478) In step S**2005**, IBC mode is disabled in response to the size of the current block satisfying the third condition (in other words, in response to the size of the current block not satisfying the fourth condition). In other words, IBC mode is not be able to be selected when both of width and height are equal to 128. The current block may be decoded using another inter mode or an intra mode. It is not necessary to decode a flag indicating whether IBC mode is used for the current block.
(479) As discussed above, IBC mode is a mode for deriving a predicted image by referring to the decoded area of the current picture.
(480) One of skill in the art will recognize that other size conditions may be employed. For example, the second value may be different value, such as 64. As another example, the second value may be same value as a size of a CTU. As another example, the second value may be same value as a size of a virtual pipeline decoding unit (VPDU).
(481) In other words, IBC mode is disabled when the current partition is a same size as a current CTU, that is, when the

current CTU is not split into smaller partitions.

(482) The present disclosure may introduce a size constraint for IBC mode, which may reduce on-chip buffer needs. This may facilitate more efficient use of resources (such as buffer area) and more efficient processing of images.

(483) [Third Aspect]

(484) FIG. **50** is a flow chart illustrating one example of a process of decoding an image block using a prediction mode selected from a plurality of prediction modes including inter mode, intra mode, and IBC mode. The prediction process as illustrated in FIG. **50** may be performed by an encoder and a decoder.

(485) In step S**3001**, a decoder determines a size of a current block. As one example, the size of the current block may be at least one of width and height of the current block. As another example, the size of the current block may be both of width and height of the current block. The decoder also may determine whether IBC mode may be employed to perform prediction for the current block. For example, the decoder may determine whether a flag indicates IBC mode is allowed to be applied to a sequence including the current block. As another example, the decoder may determine the size of the block in response to IBC mode being potentially applicable to the current block.

(486) In step S**3002** and step S**3003**, a decoder determines whether the size of the current block satisfies a size condition. For example, the size condition may be a fifth condition that one of width and height of the current block is equal to a third value and other of width and height is equal to a fourth value. The third value is a threshold value and may be 128 as an example. The fourth value is a threshold value and may be 64 as an example. In other words, the size condition is that the size of the current block is 128×64. The size condition may be a sixth condition that the size of the current block is not 128×64.

(487) In step S**3004**, in response to the size of the current block satisfying the fifth condition (in other words, in response to the size of the current block not satisfying the sixth condition), IBC mode may be applied to the current block to generate the prediction image using determined motion information, which may be determined based on the size and location of the current block, as discussed in more detail below with reference to FIG. **51**A, **51**B, **52**A, **52**B. In other words, determined motion information is used to generate prediction image of IBC mode when the size of the current block is 128×64 (or 64×128). This facilitates avoiding constraints on encoding the motion vector in the bitstream that may arise when the size of the current block satisfies a size condition, such as the fifth condition.

(488) FIGS. **51**A, **51**B, **52**A, and **52**B are conceptual diagrams illustrating examples of the determined motion information. In FIGS. **51**A and **51**B, the determined motion information is a motion vector of [−64, 0] when the size of the current block is 64×128. In FIG. **52**A, the determined motion information is a motion vector of [−128, 64] when the size of the current block is 128×64 and the current block is a top half partition of the current CTU. In FIG. **52**B, the determined motion information is a motion vector of [0, −64] when the size of the current block is 128×64 and the current block is a bottom half partition of the current CTU.

(489) As another example of step S**3004**, IBC may not be applied to the current block. In other words, the current block may be decoded using a prediction mode selected from a plurality of prediction modes including IBC mode, another inter mode or an intra mode.

(490) In step S**3005**, IBC mode may be applied to the current block to generate the prediction image using decoded motion information in response to the size of the current block not satisfying the third condition (in other words, in response to the size of the current block satisfying the fourth condition). In other words, motion information decoded from the bitstream is used for IBC mode when the size of the current block is not 128×64.

(491) The third value may be different value. For example, the third value may be the same value as a size of a CTU. As another example, the third value may be the same value as a size of a VPDU.

(492) The fourth value may be different value. For example, the fourth value may be the same value as half size of a CTU. As another example, the fourth value may be the same value as half size of a VPDU.

(493) In other words, determined motion information is used for IBC mode when one side of the current block is same size as a current CTU (or VPDU) and other side of the current block is half size as the current CTU (or VPDU).

(494) One or more of the aspects disclosed herein may be performed by combining at least part of the other aspects in the present disclosure. In addition, one or more of the aspects disclosed herein may be performed by combining, with other aspects, part of the processes indicated in any of the flow charts according to the aspects, part of the configuration of any of the devices, part of syntaxes, etc.

(495) [Implementations and Applications]

(496) As described in each of the above embodiments, each functional or operational block may typically be realized as an MPU (micro processing unit) and memory, for example. Moreover, processes performed by each of the functional blocks may be realized as a program execution unit, such as a processor which reads and executes software (a program) recorded on a recording medium such as ROM. The software may be distributed. The software may be recorded on a variety of recording media such as semiconductor memory. Note that each functional block can also be realized as hardware (dedicated circuit). Various combinations of hardware and software may be employed.

(497) The processing described in each of the embodiments may be realized via integrated processing using a single apparatus (system), and, alternatively, may be realized via decentralized processing using a plurality of apparatuses. Moreover, the processor that executes the above-described program may be a single processor or a plurality of processors. In other words, integrated processing may be performed, and, alternatively, decentralized processing may be

performed.
(498) Embodiments of the present disclosure are not limited to the above exemplary embodiments; various modifications may be made to the exemplary embodiments, the results of which are also included within the scope of the embodiments of the present disclosure.
(499) Next, application examples of the moving picture encoding method (image encoding method) and the moving picture decoding method (image decoding method) described in each of the above embodiments will be described, as well as various systems that implement the application examples. Such a system may be characterized as including an image encoder that employs the image encoding method, an image decoder that employs the image decoding method, or an image encoder-decoder that includes both the image encoder and the image decoder. Other configurations of such a system may be modified on a case-by-case basis.
(500) [Usage Examples]
(501) FIG. **53** illustrates an overall configuration of content providing system ex**100** suitable for implementing a content distribution service. The area in which the communication service is provided is divided into cells of desired sizes, and base stations ex**106**, ex**107**, ex**108**, ex**109**, and ex**110**, which are fixed wireless stations in the illustrated example, are located in respective cells.
(502) In content providing system ex**100**, devices including computer ex**111**, gaming device ex**112**, camera ex**113**, home appliance ex**114**, and smartphone ex**115** are connected to internet ex**101** via internet service provider ex**102** or communications network ex**104** and base stations ex**106** through ex**110**. Content providing system ex**100** may combine and connect any combination of the above devices. In various implementations, the devices may be directly or indirectly connected together via a telephone network or near field communication, rather than via base stations ex**106** through ex**110**. Further, streaming server ex**103** may be connected to devices including computer ex**111**, gaming device ex**112**, camera ex**113**, home appliance ex**114**, and smartphone ex**115** via, for example, Internet ex**101**. Streaming server ex**103** may also be connected to, for example, a terminal in a hotspot in airplane ex**117** via satellite ex**116**.
(503) Note that instead of base stations ex**106** through ex**110**, wireless access points or hotspots may be used. Streaming server ex**103** may be connected to communications network ex**104** directly instead of via internet ex**101** or internet service provider ex**102**, and may be connected to airplane ex**117** directly instead of via satellite ex**116**.
(504) Camera ex**113** is a device capable of capturing still images and video, such as a digital camera. Smartphone ex**115** is a smartphone device, cellular phone, or personal handy-phone system (PHS) phone that can operate under the mobile communications system standards of the 2G, 3G, 3.9G, and 4G systems, as well as the next-generation 5G system.
(505) Home appliance ex**114** is, for example, a refrigerator or a device included in a home fuel cell cogeneration system.
(506) In content providing system ex**100**, a terminal including an image and/or video capturing function is capable of, for example, live streaming by connecting to streaming server ex**103** via, for example, base station ex**106**. When live streaming, a terminal (e.g., computer ex**111**, gaming device ex**112**, camera ex**113**, home appliance ex**114**, smartphone ex**115**, or a terminal in airplane ex**117**) may perform the encoding processing described in the above embodiments on still-image or video content captured by a user via the terminal, may multiplex video data obtained via the encoding and audio data obtained by encoding audio corresponding to the video, and may transmit the obtained data to streaming server ex**103**. In other words, the terminal functions as the image encoder according to one aspect of the present disclosure.
(507) Streaming server ex**103** streams transmitted content data to clients that request the stream. Client examples include computer ex**111**, gaming device ex**112**, camera ex**113**, home appliance ex**114**, smartphone ex**115**, and terminals inside airplane ex**117**, which are capable of decoding the above-described encoded data. Devices that receive the streamed data may decode and reproduce the received data. In other words, the devices may each function as the image decoder, according to one aspect of the present disclosure.
(508) [Decentralized Processing]
(509) Streaming server ex**103** may be realized as a plurality of servers or computers between which tasks such as the processing, recording, and streaming of data are divided. For example, streaming server ex**103** may be realized as a content delivery network (CDN) that streams content via a network connecting multiple edge servers located throughout the world. In a CDN, an edge server physically near the client may be dynamically assigned to the client. Content is cached and streamed to the edge server to reduce load times. In the event of, for example, some type of error or change in connectivity due, for example, to a spike in traffic, it is possible to stream data stably at high speeds, since it is possible to avoid affected parts of the network by, for example, dividing the processing between a plurality of edge servers, or switching the streaming duties to a different edge server and continuing streaming.
(510) Decentralization is not limited to just the division of processing for streaming; the encoding of the captured data may be divided between and performed by the terminals, on the server side, or both. In one example, in typical encoding, the processing is performed in two loops. The first loop is for detecting how complicated the image is on a frame-by-frame or scene-by-scene basis, or detecting the encoding load. The second loop is for processing that maintains image quality and improves encoding efficiency. For example, it is possible to reduce the processing load of the terminals and improve the quality and encoding efficiency of the content by having the terminals perform the first loop of the encoding and having the server side that received the content perform the second loop of the encoding. In

such a case, upon receipt of a decoding request, it is possible for the encoded data resulting from the first loop performed by one terminal to be received and reproduced on another terminal in approximately real time. This makes it possible to realize smooth, real-time streaming.
(511) In another example, camera ex**113** or the like extracts a feature amount (an amount of features or characteristics) from an image, compresses data related to the feature amount as metadata, and transmits the compressed metadata to a server. For example, the server determines the significance of an object based on the feature amount, and changes the quantization accuracy accordingly to perform compression suitable for the meaning (or content significance) of the image. Feature amount data is particularly effective in improving the precision and efficiency of motion vector prediction during the second compression pass performed by the server. Moreover, encoding that has a relatively low processing load, such as variable length coding (VLC), may be handled by the terminal, and encoding that has a relatively high processing load, such as context-adaptive binary arithmetic coding (CABAC), may be handled by the server.
(512) In yet another example, there are instances in which a plurality of videos of approximately the same scene are captured by a plurality of terminals in, for example, a stadium, shopping mall, or factory. In such a case, for example, the encoding may be decentralized by dividing processing tasks between the plurality of terminals that captured the videos and, if necessary, other terminals that did not capture the videos, and the server, on a per-unit basis. The units may be, for example, groups of pictures (GOP), pictures, or tiles resulting from dividing a picture. This makes it possible to reduce load times and achieve streaming that is closer to real time.
(513) Since the videos are of approximately the same scene, management and/or instructions may be carried out by the server so that the videos captured by the terminals can be cross-referenced. Moreover, the server may receive encoded data from the terminals, change the reference relationship between items of data, or correct or replace pictures themselves, and then perform the encoding. This makes it possible to generate a stream with increased quality and efficiency for the individual items of data.
(514) Furthermore, the server may stream video data after performing transcoding to convert the encoding format of the video data. For example, the server may convert the encoding format from MPEG to VP (e.g., VP9), may convert H.264 to H.265, etc.
(515) In this way, encoding can be performed by a terminal or one or more servers. Accordingly, although the device that performs the encoding is referred to as a "server" or "terminal" in the following description, some or all of the processes performed by the server may be performed by the terminal, and likewise some or all of the processes performed by the terminal may be performed by the server. This also applies to decoding processes.
(516) [3D, Multi-Angle]
(517) There has been an increase in usage of images or videos combined from images or videos of different scenes concurrently captured, or of the same scene captured from different angles, by a plurality of terminals such as camera ex**113** and/or smartphone ex**115**. Videos captured by the terminals may be combined based on, for example, the separately obtained relative positional relationship between the terminals, or regions in a video having matching feature points.
(518) In addition to the encoding of two-dimensional moving pictures, the server may encode a still image based on scene analysis of a moving picture, either automatically or at a point in time specified by the user, and transmit the encoded still image to a reception terminal. Furthermore, when the server can obtain the relative positional relationship between the video capturing terminals, in addition to two-dimensional moving pictures, the server can generate three-dimensional geometry of a scene based on video of the same scene captured from different angles. The server may separately encode three-dimensional data generated from, for example, a point cloud and, based on a result of recognizing or tracking a person or object using three-dimensional data, may select or reconstruct and generate a video to be transmitted to a reception terminal, from videos captured by a plurality of terminals.
(519) This allows the user to enjoy a scene by freely selecting videos corresponding to the video capturing terminals, and allows the user to enjoy the content obtained by extracting a video at a selected viewpoint from three-dimensional data reconstructed from a plurality of images or videos. Furthermore, as with video, sound may be recorded from relatively different angles, and the server may multiplex audio from a specific angle or space with the corresponding video, and transmit the multiplexed video and audio.
(520) In recent years, content that is a composite of the real world and a virtual world, such as virtual reality (VR) and augmented reality (AR) content, has also become popular. In the case of VR images, the server may create images from the viewpoints of both the left and right eyes, and perform encoding that tolerates reference between the two viewpoint images, such as multi-view coding (MVC), and, alternatively, may encode the images as separate streams without referencing. When the images are decoded as separate streams, the streams may be synchronized when reproduced, so as to recreate a virtual three-dimensional space in accordance with the viewpoint of the user.
(521) In the case of AR images, the server may superimpose virtual object information existing in a virtual space onto camera information representing a real-world space, based on a three-dimensional position or movement from the perspective of the user. The decoder may obtain or store virtual object information and three-dimensional data, generate two-dimensional images based on movement from the perspective of the user, and then generate superimposed data by seamlessly connecting the images. Alternatively, the decoder may transmit, to the server, motion from the perspective of

the user in addition to a request for virtual object information. The server may generate superimposed data based on three-dimensional data stored in the server in accordance with the received motion, and encode and stream the generated superimposed data to the decoder. Note that superimposed data typically includes, in addition to RGB values, an α value indicating transparency, and the server sets the α value for sections other than the object generated from three-dimensional data to, for example, 0, and may perform the encoding while those sections are transparent. Alternatively, the server may set the background to a determined RGB value, such as a chroma key, and generate data in which areas other than the object are set as the background. The determined RGB value may be predetermined.
(522) Decoding of similarly streamed data may be performed by the client (e.g., the terminals), on the server side, or be divided therebetween. In one example, one terminal may transmit a reception request to a server, the requested content may be received and decoded by another terminal, and a decoded signal may be transmitted to a device having a display. It is possible to reproduce high image quality data by decentralizing processing and appropriately selecting content regardless of the processing ability of the communications terminal itself. In yet another example, while a TV, for example, is receiving image data that is large in size, a region of a picture, such as a tile obtained by dividing the picture, may be decoded and displayed on a personal terminal or terminals of a viewer or viewers of the TV. This makes it possible for the viewers to share a big-picture view as well as for each viewer to check his or her assigned area, or inspect a region in further detail up close.
(523) In situations in which a plurality of wireless connections are possible over near, mid, and far distances, indoors or outdoors, it may be possible to seamlessly receive content using a streaming system standard such as MPEG-DASH. The user may switch between data in real time while freely selecting a decoder or display apparatus including the user's terminal, displays arranged indoors or outdoors, etc. Moreover, using, for example, information on the position of the user, decoding can be performed while switching which terminal handles decoding and which terminal handles the displaying of content. This makes it possible to map and display information, while the user is on the move in route to a destination, on the wall of a nearby building in which a device capable of displaying content is embedded, or on part of the ground. Moreover, it is also possible to switch the bit rate of the received data based on the accessibility to the encoded data on a network, such as when encoded data is cached on a server quickly accessible from the reception terminal, or when encoded data is copied to an edge server in a content delivery service.
(524) [Scalable Encoding]
(525) The switching of content will be described with reference to a scalable stream, illustrated in FIG. **54**, which is compression coded via implementation of the moving picture encoding method described in the above embodiments. The server may have a configuration in which content is switched while making use of the temporal and/or spatial scalability of a stream, which is achieved by division into and encoding of layers, as illustrated in FIG. **54**. Note that there may be a plurality of individual streams that are of the same content but different quality. In other words, by determining which layer to decode based on internal factors, such as the processing ability on the decoder side, and external factors, such as communication bandwidth, the decoder side can freely switch between low resolution content and high resolution content while decoding. For example, in a case in which the user wants to continue watching, for example at home on a device such as a TV connected to the internet, a video that the user had been previously watching on smartphone ex**115** while on the move, the device can simply decode the same stream up to a different layer, which reduces the server side load.
(526) Furthermore, in addition to the configuration described above, in which scalability is achieved as a result of the pictures being encoded per layer, with the enhancement layer being above the base layer, the enhancement layer may include metadata based on, for example, statistical information on the image. The decoder side may generate high image quality content by performing super-resolution imaging on a picture in the base layer based on the metadata. Super-resolution imaging may improve the SN ratio while maintaining resolution and/or increasing resolution. Metadata includes information for identifying a linear or a non-linear filter coefficient, as used in super-resolution processing, or information for identifying a parameter value in filter processing, or machine learning, or a least squares method used in super-resolution processing.
(527) Alternatively, a configuration may be provided in which a picture is divided into, for example, tiles in accordance with, for example, the meaning of an object in the image. On the decoder side, only a partial region is decoded by selecting a tile to decode. Further, by storing an attribute of the object (person, car, ball, etc.) and a position of the object in the video (coordinates in identical images) as metadata, the decoder side can identify the position of a desired object based on the metadata, and determine which tile or tiles include that object. For example, as illustrated in FIG. **55**, metadata may be stored using a data storage structure different from pixel data, such as an SEI (supplemental enhancement information) message in HEVC. This metadata indicates, for example, the position, size, or color of the main object.
(528) Metadata may be stored in units of a plurality of pictures, such as stream, sequence, or random access units. The decoder side can obtain, for example, the time at which a specific person appears in the video, and by fitting the time information with picture unit information, can identify a picture in which the object is present, and can determine the position of the object in the picture.
(529) [Web Page Optimization]
(530) FIG. **56** illustrates an example of a display screen of a web page on computer ex**111**, for example. FIG. **57**

illustrates an example of a display screen of a web page on smartphone ex**115**, for example. As illustrated in FIG. **56** and FIG. **57**, a web page may include a plurality of image links that are links to image content, and the appearance of the web page may differ depending on the device used to view the web page. When a plurality of image links are viewable on the screen, until the user explicitly selects an image link, or until the image link is in the approximate center of the screen or the entire image link fits in the screen, the display apparatus (decoder) may display, as the image links, still images included in the content or I pictures; may display video such as an animated gif using a plurality of still images or I pictures; or may receive only the base layer, and decode and display the video.
(531) When an image link is selected by the user, the display apparatus performs decoding while, for example, giving the highest priority to the base layer. Note that if there is information in the HTML code of the web page indicating that the content is scalable, the display apparatus may decode up to the enhancement layer. Further, in order to guarantee real-time reproduction, before a selection is made or when the bandwidth is severely limited, the display apparatus can reduce delay between the point in time at which the leading picture is decoded and the point in time at which the decoded picture is displayed (that is, the delay between the start of the decoding of the content to the displaying of the content) by decoding and displaying only forward reference pictures (I picture, P picture, forward reference B picture). Still further, the display apparatus may purposely ignore the reference relationship between pictures, and coarsely decode all B and P pictures as forward reference pictures, and then perform normal decoding as the number of pictures received over time increases.
(532) [Autonomous Driving]
(533) When transmitting and receiving still image or video data such as two- or three-dimensional map information for autonomous driving or assisted driving of an automobile, the reception terminal may receive, in addition to image data belonging to one or more layers, information on, for example, the weather or road construction as metadata, and associate the metadata with the image data upon decoding. Note that metadata may be assigned per layer and, alternatively, may simply be multiplexed with the image data.
(534) In such a case, since the automobile, drone, airplane, etc., containing the reception terminal is mobile, the reception terminal may seamlessly receive and perform decoding while switching between base stations among base stations ex**106** through ex**110** by transmitting information indicating the position of the reception terminal. Moreover, in accordance with the selection made by the user, the situation of the user, and/or the bandwidth of the connection, the reception terminal may dynamically select to what extent the metadata is received, or to what extent the map information, for example, is updated.
(535) In content providing system ex**100**, the client may receive, decode, and reproduce, in real time, encoded information transmitted by the user.
(536) [Streaming of Individual Content]
(537) In content providing system ex**100**, in addition to high image quality, long content distributed by a video distribution entity, unicast or multicast streaming of low image quality, and short content from an individual are also possible. Such content from individuals is likely to further increase in popularity. The server may first perform editing processing on the content before the encoding processing, in order to refine the individual content. This may be achieved using the following configuration, for example.
(538) In real time while capturing video or image content, or after the content has been captured and accumulated, the server performs recognition processing based on the raw data or encoded data, such as capture error processing, scene search processing, meaning analysis, and/or object detection processing. Then, based on the result of the recognition processing, the server—either when prompted or automatically—edits the content, examples of which include: correction such as focus and/or motion blur correction; removing low-priority scenes such as scenes that are low in brightness compared to other pictures, or out of focus; object edge adjustment; and color tone adjustment. The server encodes the edited data based on the result of the editing. It is known that excessively long videos tend to receive fewer views. Accordingly, in order to keep the content within a specific length that scales with the length of the original video, the server may, in addition to the low-priority scenes described above, automatically clip out scenes with low movement, based on an image processing result. Alternatively, the server may generate and encode a video digest based on a result of an analysis of the meaning of a scene.
(539) There may be instances in which individual content may include content that infringes a copyright, moral right, portrait rights, etc. Such instance may lead to an unfavorable situation for the creator, such as when content is shared beyond the scope intended by the creator. Accordingly, before encoding, the server may, for example, edit images so as to blur faces of people in the periphery of the screen or blur the inside of a house, for example. Further, the server may be configured to recognize the faces of people other than a registered person in images to be encoded, and when such faces appear in an image, may apply a mosaic filter, for example, to the face of the person. Alternatively, as pre- or post-processing for encoding, the user may specify, for copyright reasons, a region of an image including a person or a region of the background to be processed. The server may process the specified region by, for example, replacing the region with a different image, or blurring the region. If the region includes a person, the person may be tracked in the moving picture, and the person's head region may be replaced with another image as the person moves.
(540) Since there is a demand for real-time viewing of content produced by individuals, which tends to be small in data size, the decoder may first receive the base layer as the highest priority, and perform decoding and reproduction,

although this may differ depending on bandwidth. When the content is reproduced two or more times, such as when the decoder receives the enhancement layer during decoding and reproduction of the base layer, and loops the reproduction, the decoder may reproduce a high image quality video including the enhancement layer. If the stream is encoded using such scalable encoding, the video may be low quality when in an unselected state or at the start of the video, but it can offer an experience in which the image quality of the stream progressively increases in an intelligent manner. This is not limited to just scalable encoding; the same experience can be offered by configuring a single stream from a low quality stream reproduced for the first time and a second stream encoded using the first stream as a reference.
(541) [Other Implementation and Application Examples]
(542) The encoding and decoding may be performed by LSI (large scale integration circuitry) ex**500** (see FIG. **53**), which is typically included in each terminal. LSI ex**500** may be configured from a single chip or a plurality of chips. Software for encoding and decoding moving pictures may be integrated into some type of a recording medium (such as a CD-ROM, a flexible disk, or a hard disk) that is readable by, for example, computer ex**111**, and the encoding and decoding may be performed using the software. Furthermore, when smartphone ex**115** is equipped with a camera, the video data obtained by the camera may be transmitted. In this case, the video data may be coded by LSI ex**500** included in smartphone ex**115**.
(543) Note that LSI ex**500** may be configured to download and activate an application. In such a case, the terminal first determines whether it is compatible with the scheme used to encode the content, or whether it is capable of executing a specific service. When the terminal is not compatible with the encoding scheme of the content, or when the terminal is not capable of executing a specific service, the terminal may first download a codec or application software and then obtain and reproduce the content.
(544) Aside from the example of content providing system ex**100** that uses internet ex**101**, at least the moving picture encoder (image encoder) or the moving picture decoder (image decoder) described in the above embodiments may be implemented in a digital broadcasting system. The same encoding processing and decoding processing may be applied to transmit and receive broadcast radio waves superimposed with multiplexed audio and video data using, for example, a satellite, even though this is geared toward multicast, whereas unicast is easier with content providing system ex**100**.
(545) [Hardware Configuration]
(546) FIG. **58** illustrates further details of smartphone ex**115** shown in FIG. **53**. FIG. **59** illustrates a configuration example of smartphone ex**115**. Smartphone ex**115** includes antenna ex**450** for transmitting and receiving radio waves to and from base station ex**110**, camera ex**465** capable of capturing video and still images, and display ex**458** that displays decoded data, such as video captured by camera ex**465** and video received by antenna ex**450**. Smartphone ex**115** further includes user interface ex**466** such as a touch panel; audio output unit ex**457** such as a speaker for outputting speech or other audio; audio input unit ex**456** such as a microphone for audio input; memory ex**467** capable of storing decoded data such as captured video or still images, recorded audio, received video or still images, and mail, as well as decoded data; and slot ex**464** which is an interface for SIM ex**468** for authorizing access to a network and various data. Note that external memory may be used instead of memory ex**467**.
(547) Main controller ex**460**, which may comprehensively control display ex**458** and user interface ex**466**, power supply circuit ex**461**, user interface input controller ex**462**, video signal processor ex**455**, camera interface ex**463**, display controller ex**459**, modulator/demodulator ex**452**, multiplexer/demultiplexer ex**453**, audio signal processor ex**454**, slot ex**464**, and memory ex**467** are connected via bus ex**470**.
(548) When the user turns on the power button of power supply circuit ex**461**, smartphone ex**115** is powered on into an operable state, and each component is supplied with power from a battery pack.
(549) Smartphone ex**115** performs processing for, for example, calling and data transmission, based on control performed by main controller ex**460**, which includes a CPU, ROM, and RAM. When making calls, an audio signal recorded by audio input unit ex**456** is converted into a digital audio signal by audio signal processor ex**454**, to which spread spectrum processing is applied by modulator/demodulator ex**452** and digital-analog conversion, and frequency conversion processing is applied by transmitter/receiver ex**451**, and the resulting signal is transmitted via antenna ex**450**. The received data is amplified, frequency converted, and analog-digital converted, inverse spread spectrum processed by modulator/demodulator ex**452**, converted into an analog audio signal by audio signal processor ex**454**, and then output from audio output unit ex**457**. In data transmission mode, text, still-image, or video data may be transmitted under control of main controller ex**460** via user interface input controller ex**462** based on operation of user interface ex**466** of the main body, for example. Similar transmission and reception processing is performed. In data transmission mode, when sending a video, still image, or video and audio, video signal processor ex**455** compression encodes, via the moving picture encoding method described in the above embodiments, a video signal stored in memory ex**467** or a video signal input from camera ex**465**, and transmits the encoded video data to multiplexer/demultiplexer ex**453**. Audio signal processor ex**454** encodes an audio signal recorded by audio input unit ex**456** while camera ex**465** is capturing a video or still image, and transmits the encoded audio data to multiplexer/demultiplexer ex**453**. Multiplexer/demultiplexer ex**453** multiplexes the encoded video data and encoded audio data using a determined scheme, modulates and converts the data using modulator/demodulator (modulator/demodulator circuit) ex**452** and transmitter/receiver ex**451**, and transmits the result via antenna ex**450**. The determined scheme may be predetermined.
(550) When video appended in an email or a chat, or a video linked from a web page, is received, for example, in order

to decode the multiplexed data received via antenna ex**450**, multiplexer/demultiplexer ex**453** demultiplexes the multiplexed data to divide the multiplexed data into a bitstream of video data and a bitstream of audio data, supplies the encoded video data to video signal processor ex**455** via synchronous bus ex**470**, and supplies the encoded audio data to audio signal processor ex**454** via synchronous bus ex**470**. Video signal processor ex**455** decodes the video signal using a moving picture decoding method corresponding to the moving picture encoding method described in the above embodiments, and video or a still image included in the linked moving picture file is displayed on display ex**458** via display controller ex**459**. Audio signal processor ex**454** decodes the audio signal and outputs audio from audio output unit ex**457**. Since real-time streaming is becoming increasingly popular, there may be instances in which reproduction of the audio may be socially inappropriate, depending on the user's environment. Accordingly, as an initial value, a configuration in which only video data is reproduced, i.e., the audio signal is not reproduced, may be preferable; audio may be synchronized and reproduced only when an input, such as when the user clicks video data, is received.
(551) Although smartphone ex**115** was used in the above example, other implementations are conceivable: a transceiver terminal including both an encoder and a decoder; a transmitter terminal including only an encoder; and a receiver terminal including only a decoder. In the description of the digital broadcasting system, an example is given in which multiplexed data obtained as a result of video data being multiplexed with audio data is received or transmitted. The multiplexed data, however, may be video data multiplexed with data other than audio data, such as text data related to the video. Further, the video data itself rather than multiplexed data may be received or transmitted.
(552) Although main controller ex**460** including a CPU is described as controlling the encoding or decoding processes, various terminals often include GPUs. Accordingly, a configuration is acceptable in which a large area is processed at once by making use of the performance ability of the GPU via memory shared by the CPU and GPU, or memory including an address that is managed so as to allow common usage by the CPU and GPU. This makes it possible to shorten encoding time, maintain the real-time nature of the stream, and reduce delay. In particular, processing relating to motion estimation, deblocking filtering, sample adaptive offset (SAO), and transformation/quantization can be effectively carried out by the GPU instead of the CPU in units of pictures, for example, all at once.

Claims

1. An image encoder, comprising: circuitry; and memory coupled to the circuitry; wherein the circuitry, in operation, performs the following: in response to a size of a block satisfying a size condition, generating a prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode, the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block; in response to the size of the block not satisfying the size condition, generating the prediction image using a prediction mode selected from the plurality of prediction modes excluding the first prediction mode; and encoding the block using the prediction image, wherein the first prediction mode is an intra block copy mode.
2. An image decoder, comprising: circuitry; and memory coupled to the circuitry; wherein the circuitry, in operation, performs the following: in response to a size of a block satisfying a size condition, generating a prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode, the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block; in response to the size of the block not satisfying the size condition, generating the prediction image using a prediction mode selected from the plurality of prediction modes excluding the first prediction mode; and decoding the block using the prediction image, wherein the first prediction mode is an intra block copy mode.
3. An image encoding method, comprising: encoding a block, the encoding including: in response to a size of the block satisfying a size condition, generating a prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode, the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block; in response to the size of the block not satisfying the size condition, generating the prediction image using a prediction mode selected from the plurality of prediction modes excluding the first prediction mode; and encoding the block using the prediction image, wherein the first prediction mode is an intra block copy mode.
4. An image decoding method, comprising: decoding a block, the decoding including: in response to a size of the block satisfying a size condition, generating a prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode, the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block; in response to the size of the block not satisfying the size condition, generating the prediction image using a prediction mode selected from the plurality of prediction modes excluding the first prediction mode; and decoding the block using the prediction image, wherein the first prediction mode is an intra block copy mode.
5. A non-transitory computer readable medium storing a bitstream, the bitstream comprising an encoded signal and syntax information according to which a decoder performs: in response to a size of a block satisfying a size condition, generating a prediction image using a prediction mode selected from a plurality of prediction modes including a first prediction mode, the first prediction mode including a prediction process using a motion vector and a reference block in a same picture as the block; in response to the size of the block not satisfying the size condition, generating the

prediction image using a prediction mode selected from the plurality of prediction modes excluding the first prediction mode; and decoding the block using the prediction image, wherein the first prediction mode is an intra block copy mode.